



Arm® Cortex® -A710 Core

Revision: r2p0

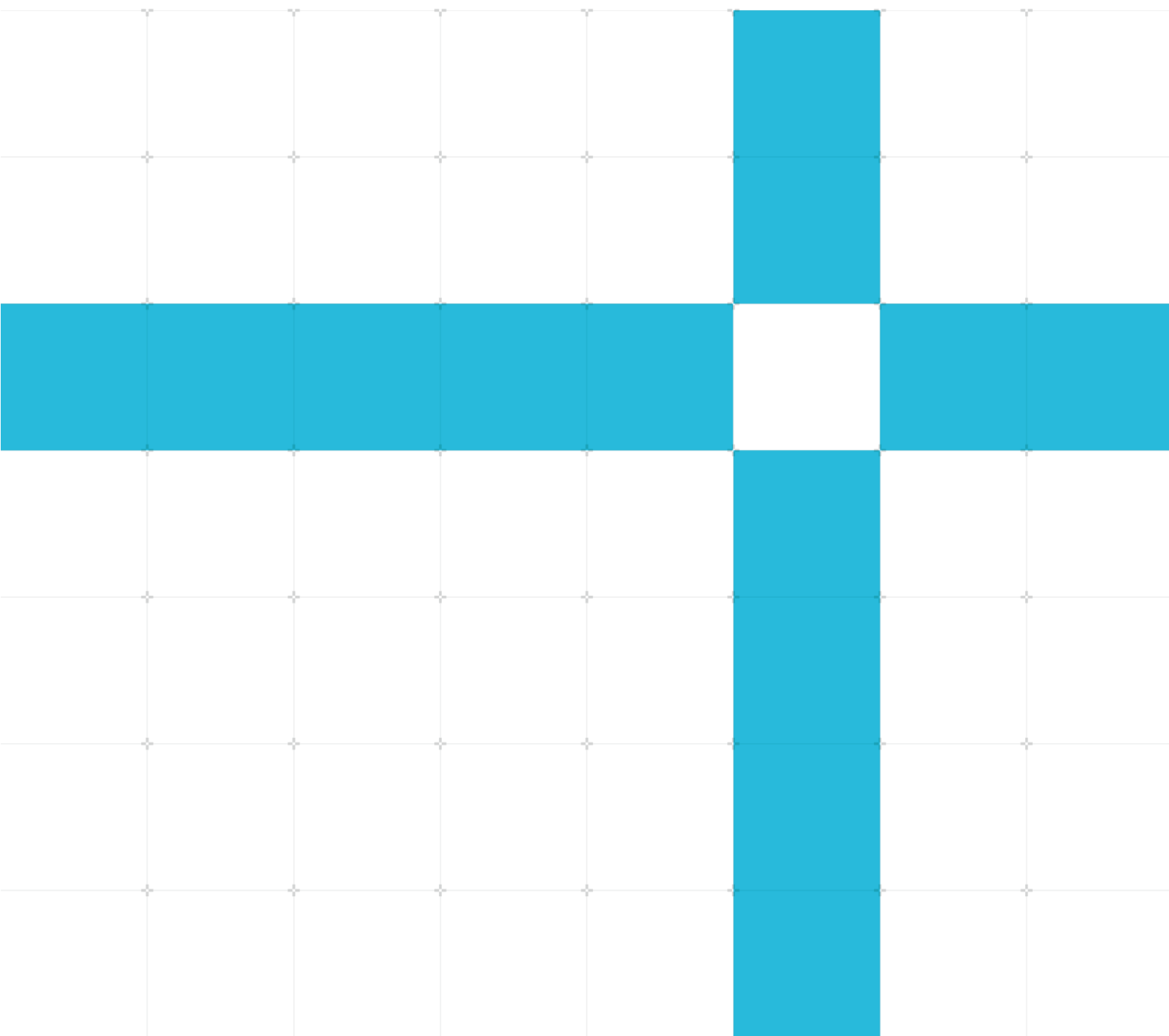
Software Optimization Guide

Non-Confidential

Copyright © 2021 Arm Limited (or its affiliates).
All rights reserved.

Issue 4.0

PJDOC-466751330-14951



Arm® Cortex® -A710 Core

Software Optimization Guide

Copyright © 2021 Arm Limited (or its affiliates). All rights reserved.

Release information

Document history

Issue	Date	Confidentiality	Change
1.0	31 Mar 2020	Confidential	First release for r0p0
2.0	15 May 2020	Confidential	Release for r1p0
3.0	21 Aug 2020	Confidential	Release for r2p0
4.0	25 May 2021	Non-Confidential	Second release for r2p0

Non-Confidential Proprietary Notice

This document is protected by copyright and other related rights and the practice or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm. No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether implementations infringe any third party patents.

THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, has undertaken no analysis to identify or understand the scope and content of, patents, copyrights, trade secrets, or other rights.

This document may include technical inaccuracies or typographical errors.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

This document consists solely of commercial items. You shall be responsible for ensuring that any use, duplication or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word "partner" in reference to Arm's customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of the Agreement shall prevail.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners. Please follow Arm's trademark usage guidelines at <https://www.arm.com/company/policies/trademarks>.

Copyright © 2021 Arm Limited (or its affiliates). All rights reserved.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

(LES-PRE-20349)

Confidentiality Status

This document is Non-Confidential. The right to use, copy and disclose this document may be subject to license restrictions in accordance with the terms of the agreement entered into by Arm and the party that Arm delivered this document to.

Unrestricted Access is an Arm internal classification.

Product Status

The information in this document is final, that is for a developed product.

Web Address

developer.arm.com

Progressive terminology commitment

Arm values inclusive communities. Arm recognizes that we and our industry have used terms that can be offensive. Arm strives to lead the industry and create change.

This document includes terms that can be offensive. We will replace these terms in a future issue of this document. If you find offensive terms in this document, please email terms@arm.com.

Contents

1 Introduction	7
1.1 Product revision status	7
1.2 Intended audience	7
1.3 Scope.....	7
1.4 Conventions.....	7
1.4.1 Glossary	7
1.4.2 Terms and abbreviations.....	8
1.4.3 Typographical conventions	9
1.5 Additional reading	10
1.6 Feedback.....	11
1.6.1 Feedback on this product.....	11
1.6.2 Feedback on content.....	11
2 Overview	12
2.1 Pipeline overview.....	13
3 Instruction characteristics.....	15
3.1 Instruction tables	15
3.2 Legend for reading the utilized pipelines	15
3.3 Branch instructions	16
3.4 Arithmetic and logical instructions.....	17
3.5 Move and shift instructions	20
3.6 Divide and multiply instructions.....	21
3.7 Saturating and parallel arithmetic instructions	24
3.8 Pointer Authentication Instructions.....	25
3.9 Miscellaneous data-processing instructions.....	27
3.10 Load instructions.....	28
3.11 Store instructions	32
3.12 Tag Load Instructions	34
3.13 Tag Store instructions	34

3.14 FP data processing instructions.....	35
3.15 FP miscellaneous instructions	37
3.16 FP load instructions.....	38
3.17 FP store instructions.....	40
3.18 ASIMD integer instructions	42
3.19 ASIMD floating-point instructions.....	46
3.20 ASIMD BFloat16 (BF16) instructions	50
3.21 ASIMD miscellaneous instructions	52
3.22 ASIMD load instructions.....	55
3.23 ASIMD store instructions	58
3.24 Cryptography extensions	61
3.25 CRC.....	62
3.26 SVE Predicate instructions	63
3.27 SVE integer instructions.....	65
3.28 SVE floating-point instructions.....	72
3.29 SVE BFloat16 (BF16) instructions.....	74
3.30 SVE Load instructions.....	75
3.31 SVE Store instructions	77
3.32 SVE Miscellaneous instructions	79
3.33 SVE Cryptographic instructions	80
4 Special considerations.....	81
4.1 Dispatch constraints.....	81
4.2 Dispatch stall	81
4.3 Optimizing general-purpose register spills and fills	81
4.4 Optimizing memory routines.....	82
4.5 Load/Store alignment.....	83
4.6 AES encryption/decryption.....	83
4.7 Region based fast forwarding	84
4.8 Branch instruction alignment.....	85
4.9 FPCR self-synchronization.....	85
4.10 Special register access.....	85
4.11 Register forwarding hazards.....	87

4.12 IT blocks.....	88
4.13 Instruction fusion	88
4.14 Zero Latency MOVs.....	89
4.15 Cache maintenance operation	90
4.16 Memory Tagging - Tagging Performance.....	90
4.17 Memory Tagging - Synchronous Mode	91
4.18 Complex ASIMD and SVE instructions	91

1 Introduction

1.1 Product revision status

The *mpn* identifier indicates the revision status of the product described in this book, for example, r1p2, where:

Rm

Identifies the major revision of the product, for example, r1.

Pn

Identifies the minor revision or modification status of the product, for example, p2.

1.2 Intended audience

This document is for system designers, system integrators, and programmers who are designing or programming a System-on-Chip (SoC) that uses an Arm core.

1.3 Scope

This document describes aspects of the Cortex-A710 core micro-architecture that influence software performance. Micro-architectural detail is limited to that which is useful for software optimization.

Documentation extends only to software visible behavior of the Cortex-A710 core and not to the hardware rationale behind the behavior.

1.4 Conventions

The following subsections describe conventions used in Arm documents.

1.4.1 Glossary

The Arm Glossary is a list of terms used in Arm documentation, together with definitions for those terms. The Arm Glossary does not contain terms that are industry standard unless the Arm meaning differs from the generally accepted meaning.

See the Arm Glossary for more information: <https://developer.arm.com/glossary>.

1.4.2 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
ALU	Arithmetic and Logical Unit
ASIMD	Advanced SIMD
MOP	Macro-OPeration
μOP	Micro-OPeration
SQRT	Square Root
T32	AArch32 Thumb® instruction set
FP	Floating-point

1.4.3 Typographical conventions

Convention	Use
<i>italic</i>	Introduces citations.
bold	Highlights interface elements, such as menu names. Denotes signal names. Also used for terms in descriptive lists, where appropriate.
monospace	Denotes text that you can enter at the keyboard, such as commands, file and program names, and source code.
monospace bold	Denotes language keywords when used outside example code.
monospace <u>underline</u>	Denotes a permitted abbreviation for a command or option. You can enter the underlined text instead of the full command or option name.
<and>	Encloses replaceable terms for assembler syntax where they appear in code or code fragments. For example: <pre>MRC p15, 0, <Rd>, <CRn>, <CRm>, <Opcode_2></pre>
SMALL CAPITALS	Used in body text for a few terms that have specific technical meanings, that are defined in the Arm® Glossary. For example, IMPLEMENTATION DEFINED, IMPLEMENTATION SPECIFIC, UNKNOWN, and UNPREDICTABLE.
 Caution	This represents a recommendation which, if not followed, might lead to system failure or damage.
 Warning	This represents a requirement for the system that, if not followed, might result in system failure or damage.
 Danger	This represents a requirement for the system that, if not followed, will result in system failure or damage.
 Note	This represents an important piece of information that needs your attention.
 Tip	This represents a useful tip that might make it easier, better or faster to perform a task.
 Remember	This is a reminder of something important that relates to the information you are reading.

1.5 Additional reading

This document contains information that is specific to this product. See the following documents for other relevant information:

Table 1-1 Arm publications

Document name	Document ID	Licensee only
<i>Arm® Architecture Reference Manual, Armv8, for Armv8-A architecture profile</i>	DDI 0487	Y
<i>Arm® Cortex®-A710 Core Technical Reference Manual</i>	101430	Y

1.6 Feedback

Arm welcomes feedback on this product and its documentation.

1.6.1 Feedback on this product

If you have any comments or suggestions about this product, contact your supplier and give:

- The product name.
- The product revision or version.
- An explanation with as much information as you can provide. Include symptoms and diagnostic procedures if appropriate.

1.6.2 Feedback on content

If you have comments on content, send an email to errata@arm.com and give:

- The title Arm® Cortex®-A710 Core Software Optimization Guide.
- The number PJDOC-466751330-14951.
- If applicable, the page number(s) to which your comments refer.
- A concise explanation of your comments.

Arm also welcomes general suggestions for additions and improvements.



Arm tests the PDF only in Adobe Acrobat and Acrobat Reader and cannot guarantee the quality of the represented document when used with any other PDF reader.

2 Overview

The Cortex-A710 core is a high-performance, low-power, and constrained area product that implements the Armv9.0-A architecture and supports all previous Armv8-A architectures up to Armv8.5-A. It targets clamshell and premium high-end smartphone applications

The key features of Cortex-A710 core are:

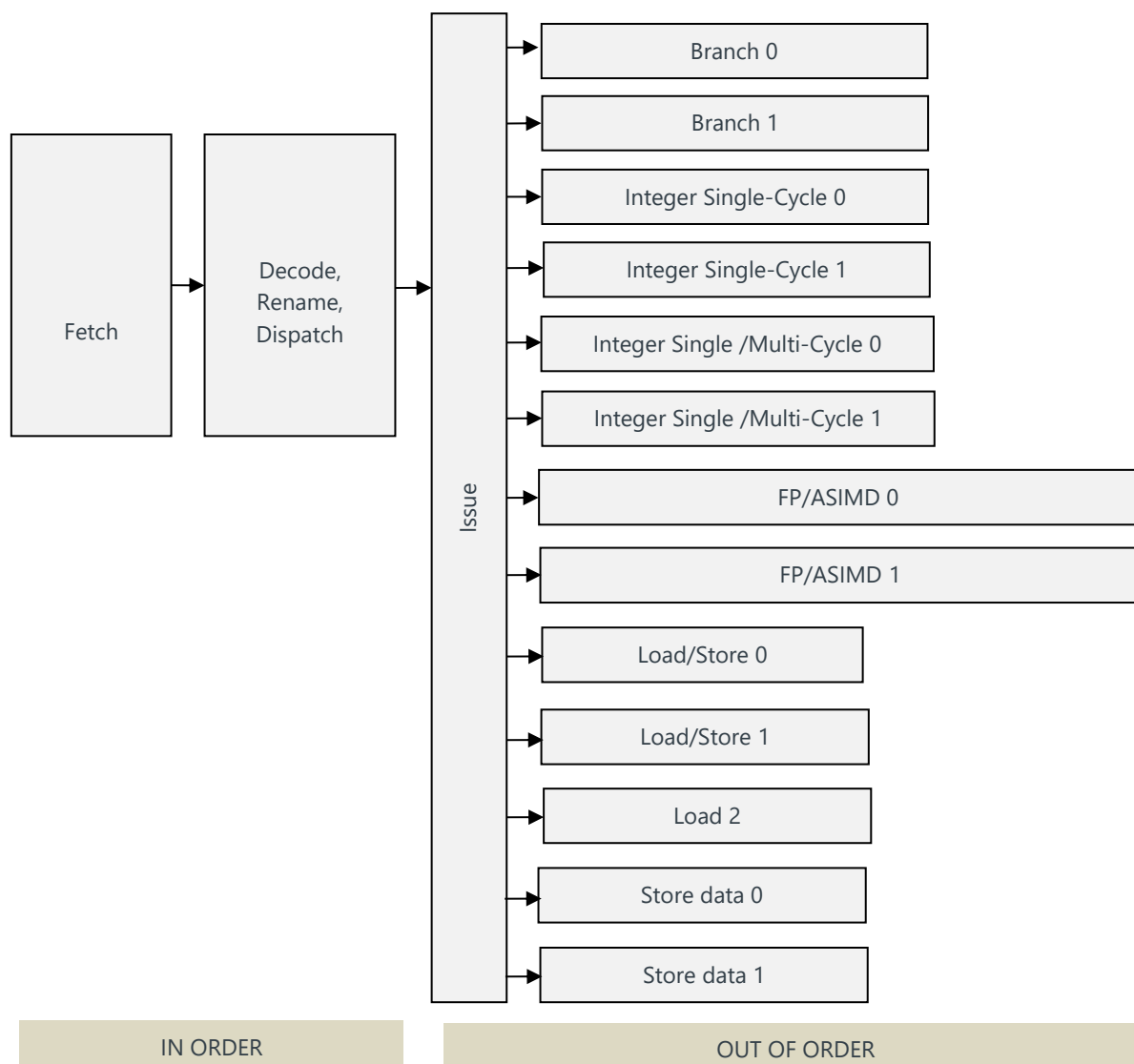
- Implementation of the Armv9-A A32, T32, and A64 instruction sets.
- AArch32 Execution state at Exception level EL0 and AArch64 Execution state at all exception levels, EL0-EL3
- Memory Management Unit (MMU)
- 40-bit Physical Address (PA) and 48-bit Virtual Address (VA)
- Generic Interrupt Controller (GIC) CPU interface to connect to an external interrupt distributor
- Generic Timers that supports 64-bit count input from an external system counter
- Implementation of the Reliability, Availability, and Serviceability (RAS) Extension
- Implementation of the Scalable Vector Extension (SVE) with a 128-bit vector length and Scalable Vector Extension 2 (SVE2)
- Integrated execution unit with Advanced SIMD and floating-point support
- Support for the optional Cryptographic Extension, which is licensed separately
- Activity Monitoring Unit (AMU)
- Separate L1 data and instruction caches
- Private, unified data and instruction L2 cache
- Support for Memory System Resource Partitioning and Monitoring (MPAM)
- Armv9-A debug logic
- Performance Monitoring Unit (PMU)
- Embedded Trace Macrocell (ETM) with support for Embedded Trace Extension (ETE)
- Trace Buffer Extension (TRBE)
- Optional Embedded Logic Analyzer (ELA)

This document describes elements of the Cortex-A710 core micro-architecture that influence software performance so that software and compilers can be optimized accordingly.

2.1 Pipeline overview

The following figure describes the high-level Cortex-A710 instruction processing pipeline. Instructions are first fetched and then decoded into internal Macro-Operations (MOPs). From there, the MOPs proceed through register renaming and dispatch stages. A MOP can be split into two Micro-Operations (μOPs) further down the pipeline after the decode stage. Once dispatched, μOPs wait for their operands and issue out-of-order to one of thirteen issue pipelines. Each issue pipeline can accept one μOP per cycle.

Figure 2-1 Cortex-A710 core pipeline



The execution pipelines support different types of operations, as shown in the following table.

Table 2-1 Cortex-A710 core operations

Instruction groups	Instructions
Branch 0/1	Branch μ OPs
Integer Single-Cycle 0/1	Integer ALU μ OPs
Integer Single/Multi-cycle 0/1	Integer shift-ALU, multiply, divide, CRC and sum-of-absolute-differences μ OPs
Load/Store 0/1	Load, Store address generation and special memory μ OPs
Load 2	Load μ OPs
Store data 0/1	Store data μ OPs
FP/ASIMD-0	ASIMD ALU, ASIMD misc, ASIMD integer multiply, FP convert, FP misc, FP add, FP multiply, FP divide, FP sqrt, crypto μ OPs, store data μ OPs
FP/ASIMD-1	ASIMD ALU, ASIMD misc, FP misc, FP add, FP multiply, ASIMD shift μ OPs, store data μ OPs, crypto μ OPs.

3 Instruction characteristics

3.1 Instruction tables

This chapter describes high-level performance characteristics for most Armv9-A instructions. A series of tables summarize the effective execution latency and throughput (instruction bandwidth per cycle), pipelines utilized, and special behaviors associated with each group of instructions. Utilized pipelines correspond to the execution pipelines described in chapter 2.

In the tables below, Exec Latency is defined as the minimum latency seen by an operation dependent on an instruction in the described group.

In the tables below, Execution Throughput is defined as the maximum throughput (in instructions per cycle) of the specified instruction group that can be achieved in the entirety of the Cortex-A710 microarchitecture.

3.2 Legend for reading the utilized pipelines

Table 3-1 Cortex-A710 core pipeline names and symbols

Pipeline name	Symbol used in tables
Branch 0/1	B
Integer single Cycle 0/1	S
Integer single Cycle 0/1 and single/multicycle 0/1	I
Integer single/multicycle 0/1	M
Integer multicycle 0	M0
Load/Store 01	L01
Load/Store 0/1 and Load 2	L
Store data 0/1	D
FP/ASIMD 0/1	V
FP/ASIMD 0	V0
FP/ASIMD 1	V1

3.3 Branch instructions

Table 3-2 AArch64 Branch instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Branch, immed	B	1	2	B	-
Branch, register	BR, RET	1	2	B	-
Branch and link, immed	BL	1	2	B, S	-
Branch and link, register	BLR	1	2	B, S	-
Compare and branch	CBZ, CBNZ, TBZ, TBNZ	1	2	B	-

Table 3-3 AArch32 Branch instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Branch, immed	B	1	2	B	-
Branch, register	BX	1	2	B	-
Branch and link, immed	BL, BLX	1	2	B, S	-
Branch and link, register	BLX	1	2	B, S	-
Compare and branch	CBZ, CBNZ	1	2	B	-

3.4 Arithmetic and logical instructions

Table 3-4 AArch64 Arithmetic and logical instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ALU, basic	ADD, ADC, AND, BIC, EON, EOR, ORN, ORR, SUB, SBC	1	4	I	-
ALU, basic, flagset	ADDs, ADCs, ANDs, BICs, SUBs, SBCs	1	3	I	-
ALU, extend and shift	ADD{S}, SUB{S}	2	2	M	-
Arithmetic, LSL shift, shift ≤ 4	ADD, SUB	1	4	I	-
Arithmetic, flagset, LSL shift, shift ≤ 4	ADDs, SUBs	1	4	I	-
Arithmetic, LSR/ASR/ROR shift or LSL shift > 4	ADD{S}, SUB{S}	2	2	M	-
Arithmetic, immediate to logical address tag	ADDG, SUBG	2	2	M	-
Conditional compare	CCMN, CCMP	1	4	I	-
Conditional select	CSEL, CSINC, CSINV, CSNEG	1	4	I	-
Convert floating-point condition flags	AXFLAG, XAFLAG	1	1	I	-
Flag manipulation instructions	SETF8, SETF16, RMIF, CFINV	1	1	I	-
Insert Random Tags	IRG	2, 3	2, 1	M, M0	1
Insert Tag Mask	GMI	1	4	I	-
Logical, shift, no flagset	AND, BIC, EON, EOR, ORN, ORR	1	4	I	-
Logical, shift, flagset	ANDs, BICs	2	2	M	-
Subtract Pointer	SUBP	1	4	I	-
Subtract Pointer, flagset	SUBPS	1	3	I	-

Table 3-5 AArch32 Arithmetic and logical instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ALU, basic, unconditional, no flagset	ADD, ADC, ADR, AND, BIC, EOR, ORN, ORR, RSB, RSC, SUB, SBC	1	4	I	-
ALU, basic, unconditional, flagset	ADDS, ADCS, ANDS, BICS, CMN, CMP, EORS, ORNS, ORRS, RSBS, RSCS, SUBS, SBCS, TEQ, TST	1	3	I	-
ALU, basic, conditional	ADD{S}, ADC{S}, AND{S}, BIC{S}, CMN, CMP, EOR{S}, ORN{S}, ORR{S}, RSB{S}, RSC{S}, SUB{S}, SBC{S}, TEQ, TST	1	1	M0	-
ALU, basic, shift by register, conditional	(same as ALU basic, conditional)	2	1	I, M0	-
ALU, basic, shift by register, unconditional, flagset	(same as ALU, basic, unconditional, flagset)	2	1	M0	-
Arithmetic, shift by register, unconditional, no flagset	ADD, ADC, RSB, RSC, SUB, SBC	2	1	M0	-
Logical, shift by register, unconditional, no flagset	AND, BIC, EOR, ORN, ORR	1	1	M0	-
Arithmetic, LSL shift by immed, shift ≤ 4 , unconditional, no flagset	ADD, ADC, RSB, RSC, SUB, SBC	1	4	I	-
Arithmetic, LSL shift by immed, shift ≤ 4 , unconditional, flagset	ADDS, ADCS, RSBS, RSCS, SUBS, SBCS	1	4	I	-
Arithmetic, LSL shift by immed, shift ≤ 4 , conditional	ADD{S}, ADC{S}, RSB{S}, RSC{S}, SUB{S}, SBC{S}	1	1	M0	-
Arithmetic, LSR/ASR/ROR shift by immed or LSL shift by immed > 4 , unconditional	ADD{S}, ADC{S}, RSB{S}, RSC{S}, SUB{S}, SBC{S}	2	2	M	-
Arithmetic, LSR/ASR/ROR shift by immed or LSL shift by immed > 4 , conditional	ADD{S}, ADC{S}, RSB{S}, RSC{S}, SUB{S}, SBC{S}	2	1	M0	-

Copyright © 2021 Arm Limited (or its affiliates). All rights reserved.

Non-Confidential

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Logical, shift by immed, no flagset, unconditional	AND, BIC, EOR, ORN, ORR	1	4	I	-
Logical, shift by immed, no flagset, conditional	AND, BIC, EOR, ORN, ORR	1	1	M0	-
Logical, shift by immed, flagset, unconditional	ANDS, BICS, EORS, ORNS, ORRS	2	2	M	-
Logical, shift by immed, flagset, conditional	ANDS, BICS, EORS, ORNS, ORRS	2	1	M0	-
Test/Compare, shift by immed	CMN, CMP, TEQ, TST	2	2	M	-
Branch forms		+1	2	+B	2

Notes:

1. The latency is 2, throughput is 2 and utilized pipeline is M when GCR_EL1.RRND = 1. When GCR_EL1.RRND = 0, latency is 3, throughput is 1 and pipeline utilized is M0.

2. Branch forms are possible when the instruction destination register is the PC. For those cases, an additional branch μ OP is required. This adds 1 cycle to the latency.

3.5 Move and shift instructions

Table 3-6 AArch32 Move and shift instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Move, basic	MOV, MOVW, MVN	1	4	I	-
Move, basic, flagset	MOVS, MVNS	1	3	I	
Move, shift by immed, no flagset	ASR, LSL, LSR, ROR, RRX, MVN	1	4	I	-
Move, shift by immed, flagset	ASRS, LSLS, LSRS, RORS, RRXS, MVNS	2	2	M	-
Move, shift by register, no flagset, unconditional	ASR, LSL, LSR, ROR, RRX, MVN	1	4	I	-
Move, shift by register, no flagset, conditional	ASR, LSL, LSR, ROR, RRX, MVN	2	2	I	-
Move, shift by register, flagset	ASRS, LSLS, LSRS, RORS, RRXS, MVNS	2	1	M0	-
Move, top	MOVT	1	4	I	-
Move, branch forms		+1	2	+B	-

3.6 Divide and multiply instructions

Table 3-7 AArch64 Divide and multiply instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Divide, W-form	SDIV, UDIV	5 to 12	1/12 to 1/5	M0	1
Divide, X-form	SDIV, UDIV	5 to 20	1/20 to 1/5	M0	1
Multiply	MUL, MNEG	2	2	M	-
Multiply accumulate, W-form	MADD, MSUB	2(1)	1	M0	2
Multiply accumulate, X-form	MADD, MSUB	2(1)	1	M0	2
Multiply accumulate long	SMADDL, SMSUBL, UMADDL, UMSUBL	2(1)	1	M0	2
Multiply high	SMULH, UMULH	3	2	M	2
Multiply long	SMNEGL, SMULL, UMNEGL, UMULL	2	2	M	-

Table 3-8 AArch32 Divide and multiply instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Divide	SDIV, UDIV	5 to 12	1/12 to 1/5	M0	1
Multiply, unconditional	MUL, SMULBB, SMULBT, SMULTB, SMULTT, SMULWB, SMULWT, SMMUL{R}, SMUAD{X}, SMUSD{X}	2	2	M	-
Multiply, conditional	MUL, SMULBB, SMULBT, SMULTB, SMULTT, SMULWB, SMULWT, SMMUL{R}, SMUAD{X}, SMUSD{X}	2	1	M0	-

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Multiply accumulate, conditional	MLA, MLS, SMLABB, SMLABT, SMLATB, SMLATT, SMLAWB, SMLAWT, SMLAD{X}, SMLS{D}{X}, SMMLA{R}, SMMLS{R}	3	1	M0, I	-
Multiply accumulate, unconditional	MLA, MLS, SMLABB, SMLABT, SMLATB, SMLATT, SMLAWB, SMLAWT, SMLAD{X}, SMLS{D}{X}, SMMLA{R}, SMMLS{R}	2(1)	1	M0	2
Multiply accumulate accumulate long, conditional	UMAAL	4	1	I, M0	-
Multiply accumulate accumulate long, unconditional	UMAAL	3	1	I, M0	-
Multiply accumulate long, no flagset	SMLAL, SMLALBB, SMLALBT, SMLALTB, SMLALTT, SMLALD{X}, SMLS{D}{X}, UMLAL	3	1	M0, I	-
Multiply accumulate long, flagset	SMLAL, SMLALBB, SMLALBT, SMLALTB, SMLALTT, SMLALD{X}, SMLS{D}{X}, UMLAL	4	1	M0, I	-
Multiply long, unconditional, no flagset	SMULL, UMULL	2	2	M	-
Multiply long, unconditional, flagset	SMULLS, UMULLS	3	1	M, I	-

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Multiply long, conditional	SMULL{S}, UMULL{S}	3	1	M, I	-

Notes:

1. Integer divides are performed using an iterative algorithm and block any subsequent divide operations until complete. Early termination is possible, depending upon the data values.
2. Multiply-accumulate pipelines support late-forwarding of accumulate operands from similar μ OPs, allowing a typical sequence of multiply-accumulate μ OPs to issue one every N cycles (accumulate latency N shown in parentheses). Accumulator forwarding is not supported for consumers of 64 bit multiply high operations.

3.7 Saturating and parallel arithmetic instructions

Table 3-9 AArch32 Saturating and parallel arithmetic instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Parallel arith, unconditional	SADD16, SADD8, SSUB16, SSUB8, UADD16, UADD8, USUB16, USUB8	2	1	M	-
Parallel arith, conditional	SADD16, SADD8, SSUB16, SSUB8, UADD16, UADD8, USUB16, USUB8	2(4)	1	M0, I	1
Parallel arith with exchange, unconditional	SASX, SSAX, UASX, USAX	3	2	I, M	-
Parallel arith with exchange, conditional	SASX, SSAX, UASX, USAX	3(5)	1	I, M0	1
Parallel halving arith, unconditional	SHADD16, SHADD8, SHSUB16, SHSUB8, UHADD16, UHADD8, UHSUB16, UHSUB8	2	2	M	-
Parallel halving arith, conditional	SHADD16, SHADD8, SHSUB16, SHSUB8, UHADD16, UHADD8, UHSUB16, UHSUB8	2	1	M0	-
Parallel halving arith with exchange	SHASX, SHSAX, UHASX, UHSAX	3	1	I, M0	-
Parallel saturating arith, unconditional	QADD16, QADD8, QSUB16, QSUB8, UQADD16, UQADD8, UQSUB16, UQSUB8	2	2	M	-

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Parallel saturating arith, conditional	QADD16, QADD8, QSUB16, QSUB8, UQADD16, UQADD8, UQSUB16, UQSUB8	2	1	M0	-
Parallel saturating arith with exchange, unconditional	QASX, QSAX, UQASX, UQSAX	3	2	I, M	-
Parallel saturating arith with exchange, conditional	QASX, QSAX, UQASX, UQSAX	3	1	I, M0	-
Saturate, unconditional	SSAT, SSAT16, USAT, USAT16	2	2	M	-
Saturate, conditional	SSAT, SSAT16, USAT, USAT16	2	1	M0	-
Saturating arith, unconditional	QADD, QSUB	2	2	M	-
Saturating arith, conditional	QADD, QSUB	2	1	M0	-
Saturating doubling arith, unconditional	QDADD, QDSUB	3	1	M, M	-
Saturating doubling arith conditional	QDADD, QDSUB	3	1	M, M0	-

Notes:

1. GE-setting instructions require three extra μ OPs and two additional cycles to conditionally update the GE field (GE latency shown in parentheses).

3.8 Pointer Authentication Instructions

Table 3-10 AArch64 pointer authentication instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Authenticate data address	AUTDA, AUTDB, AUTDZA, AUTDZB	5	1	M0	-
Authenticate instruction address	AUTIA, AUTIB, AUTIA1716, AUTIB1716, AUTIASP, AUTIBSP, AUTIAZ, AUTIBZ, AUTIZA, AUTIZB	5	1	M0	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Branch and link, register, with pointer authentication	BLRAA, BLRAAZ, BLRAB, BLRABZ	6	1	M0, B	
Branch, register, with pointer authentication	BRAA, BRAAZ, BRAB, BRABZ	6	1	M0, B	
Branch, return, with pointer authentication	RETA, RETB	6	1	M0, B	
Compute pointer authentication code for data address	PACDA, PACDB, PACDZA, PACDZB	5	1	M0	
Compute pointer authentication code, using generic key	PACGA	5	1	M0	
Compute pointer authentication code for instruction address	PACIA, PACIB, PACIA1716, PACIB1716, PACIASP, PACIBSP, PACIAZ, PACIBZ, PACIZA, PACIZB	5	1	M0	
Load register, with pointer authentication	LDRAA, LDRAB	9	1	M0, L	
Strip pointer authentication code	XPACD, XPACI, XPACLRI	2	1	M0	

3.9 Miscellaneous data-processing instructions

Table 3-11 AArch64 Miscellaneous data-processing instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Address generation	ADR, ADRP	1	4	I	-
Bitfield extract, one reg	EXTR	1	4	I	-
Bitfield extract, two regs	EXTR	3	2	I, M	-
Bitfield move, basic	SBFM, UBFM	1	4	I	-
Bitfield move, insert	BFM	2	2	M	-
Count leading	CLS, CLZ	1	4	I	-
Move immed	MOVN, MOVK, MOVZ	1	4	I	-
Reverse bits/bytes	RBIT, REV, REV16, REV32	1	4	I	-
Variable shift	ASRV, LSLV, LSRV, RORV	1	4	I	-

Table 3-12 AArch32 Miscellaneous data-processing instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Bit field extract	SBFX, UBFX	1	4	I	-
Bit field insert/clear, unconditional	BFI, BFC	2	2	M	-
Bit field insert/clear, conditional	BFI, BFC	2	1	M0	-
Count leading zeros	CLZ	1	4	I	-
Pack halfword, unconditional	PKH	2	2	M	-
Pack halfword, conditional	PKH	2	1	M0	-
Reverse bits/bytes	RBIT, REV, REV16, REVSH	1	4	I	-
Select bytes, unconditional	SEL	1	4	I	-
Select bytes, conditional	SEL	2	2	I	-
Sign/zero extend, normal	SXTB, SXTH, UXTB, UXTH	1	4	I	-
Sign/zero extend, parallel, unconditional	SXTB16, UXTB16	2	2	M	-

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Sign/zero extend, parallel, conditional	SXTB16, UXTB16	2	1	M0	-
Sign/zero extend and add, normal, unconditional	SXTAB, SXTAH, UXTAB, UXTAH	2	2	M	-
Sign/zero extend and add, normal, conditional	SXTAB, SXTAH, UXTAB, UXTAH	2	1	M0	-
Sign/zero extend and add, parallel, unconditional	SXTAB16, UXTAB16	4	1	M	-
Sign/zero extend and add, parallel, conditional	SXTAB16, UXTAB16	4	1	M, M0	-
Sum of absolute differences	USAD8	2	1	M0	-
Sum of absolute differences accumulate, unconditional	USADA8	2	1	M0	-
Sum of absolute differences accumulate, conditional	USADA8	3	1	M0, I	-

3.10 Load instructions

The latencies shown assume the memory access hits in the Level 1 Data Cache and represent the maximum latency to load all the registers written by the instruction.

Table 3-13 AArch64 Load instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Load register, literal	LDR, LDRSW, PRFM	4	3	L	-
Load register, unscaled immed	LDUR, LDURB, LDURH, LDURSB, LDURSH, LDURSW, PRFUM	4	3	L	-
Load register, immed post-index	LDR, LDRB, LDRH, LDRSB, LDRSH, LDRSW	4	3	L, I	-
Load register, immed pre-index	LDR, LDRB, LDRH, LDRSB, LDRSH, LDRSW	4	3	L, I	-
Load register, immed unprivileged	LDTR, LDTRB, LDTRH, LDTRSB, LDTRSH, LDTRSW	4	3	L	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Load register, unsigned immed	LDR, LDRB, LDRH, LDRSB, LDRSH, LDRSW, PRFM	4	3	L	-
Load register, register offset, basic	LDR, LDRB, LDRH, LDRSB, LDRSH, LDRSW, PRFM	4	3	L	-
Load register, register offset, scale by 4/8	LDR, LDRSW, PRFM	4	3	L	-
Load register, register offset, scale by 2	LDRH, LDRSH	4	3	L	-
Load register, register offset, extend	LDR, LDRB, LDRH, LDRSB, LDRSH, LDRSW, PRFM	4	3	L	-
Load register, register offset, extend, scale by 4/8	LDR, LDRSW, PRFM	4	3	L	-
Load register, register offset, extend, scale by 2	LDRH, LDRSH	4	3	L	-
Load pair, signed immed offset, normal, W-form	LDP, LDNP	4	3	L	-
Load pair, signed immed offset, normal, X-form	LDP, LDNP	4	1.5	L	-
Load pair, signed immed offset, signed words	LDPSW	5	1	I, L	-
Load pair, immed post-index or immed pre-index, normal, W-form	LDP	4	3	L, I	-
Load pair, immed post-index or immed pre-index, normal, X-form	LDP	4	1.5	L, I	-
Load pair, immed post-index or immed pre-index, signed words	LDPSW	5	1	I, L	-

Table 3-14 AArch32 Load instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Load, immed offset	LDR{T}, LDRB{T}, LDRD, LDRH{T}, LDRSB{T}, LDRSH{T}	4	3	L	1, 2

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Load, register offset, plus	LDR, LDRB, LDRD, LDRH, LDRSB, LDRSH	4	3	L	1, 2
Load, register offset, minus	LDR, LDRB, LDRD, LDRH, LDRSB, LDRSH	5	3	I, L	1, 2
Load, scaled register offset, plus, LSL2	LDR, LDRB	4	3	L	1, 2
Load, scaled register offset, other	LDR, LDRB, LDRH, LDRSB, LDRSH	5	3	I, L	1, 2
Load, immed pre-indexed	LDR, LDRB, LDRD, LDRH, LDRSB, LDRSH	4	3	L, I	1, 2
Load, register pre-indexed	LDRH, LDRSB, LDRSH	5	3	I, L, M0	1, 2, 3
Load, register pre-indexed	LDRD	4	3	L, M0	1, 2, 3
Load, scaled register pre-indexed, plus, LSL2	LDR, LDRB	4	3	L, M0	1, 2, 3
Load, scaled register pre-indexed, unshifted	LDR, LDRB	4	3	L, M0	1, 2, 3
Load, scaled register pre-indexed, other	LDR, LDRB	5	3	I, L, M0	1, 2, 3
Load, immed post-indexed	LDR{T}, LDRB{T}, LDRD, LDRH{T}, LDRSB{T}, LDRSH{T}	4	3	L, I	1, 2
Load, register post-indexed	LDR{T}, LDRB{T}, LDRH{T}, LDRSB{T}, LDRSH{T}	5	3	I, L, M0	1, 2, 3
Load, register post-indexed	LDRD	4	3	L, M0	1, 2, 3
Preload, immed offset	PLD, PLDW	4	3	L	-
Preload, register offset, plus, LSL2 and unshifted	PLD, PLDW	4	3	L	-
Preload, register offset, minus	PLD, PLDW	5	3	I, L	-
Load multiple, no writeback, base reg not in list	LDMIA, LDMIB, LMDMA, LDMDB	N	3/R	L	1, 4, 5
Load multiple, no writeback, base reg in list	LDMIA, LDMIB, LMDMA, LDMDB	1 + N	3/R	I, L	1, 4, 5

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Load multiple, writeback	LDMIA, LDMIB, LDMDA, LDMDB, POP	1+ N	3/R	L, I	1, 4, 5
(Load, all branch forms)	-	+1	-	+ B	6

Notes:

1. Conditional loads have extra μOP (s) which goes down pipeline I and have 1 cycle extra latency compared to their unconditional counterparts.
2. Conditional loads go down L01 pipe and have an execution throughput of 2 whereas unconditional versions have a throughput of 3.
3. The address update op goes down pipeline 'I' if the load is unconditional.
4. N is floor $[(\text{num_reg}+5)/6]$.
5. R is floor $[(\text{num_reg} +1)/2]$.
6. Branch forms are possible when the instruction destination register is the PC. For those cases, an additional branch μOP is required. This adds 1 cycle to the latency.

3.11 Store instructions

The following table describes performance characteristics for standard store instructions. Stores μ OPs are split into address and data μ OPs. Once executed, stores are buffered and committed in the background.

Table 3-15 AArch64 Store instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Store register, unscaled immed	STUR, STURB, STURH	1	2	L01, D	-
Store register, immed post-index	STR, STRB, STRH	1	2	L01, D, I	-
Store register, immed pre-index	STR, STRB, STRH	1	2	L01, D, I	-
Store register, immed unprivileged	STTR, STTRB, STTRH	1	2	L01, D	-
Store register, unsigned immed	STR, STRB, STRH	1	2	L01, D	-
Store register, register offset, basic	STR, STRB, STRH	1	2	L01, D	-
Store register, register offset, scaled by 4/8	STR	1	2	L01, D	-
Store register, register offset, scaled by 2	STRH	1	2	I, L01, D	-
Store register, register offset, extend	STR, STRB, STRH	1	2	L01, D	-
Store register, register offset, extend, scale by 4/8	STR	1	2	L01, D	-
Store register, register offset, extend, scale by 2	STRH	1	2	I, L01, D	-
Store pair, immed offset	STP, STNP	1	2	L01, D	-
Store pair, immed post-index	STP	1	2	L01, D, I	-
Store pair, immed pre-index	STP	1	2	L01, D, I	-

Table 3-16 AArch32 Store instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Store, immed offset	STR{T}, STRB{T}, STRD, STRH{T}	1	2	L01, D	-
Store, register offset, plus	STR, STRB, STRD, STRH	1	2	L01, D	-

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Store, register offset, minus	STR, STRB, STRD, STRH	1	2	L01, D	-
Store, scaled register offset, plus, no shift	STR, STRB	1	2	L01, D	-
Store, scaled register offset, plus, LSL2	STR, STRB	1	2	L01, D	-
Store, scaled register offset, plus, other	STR, STRB	2	2	I, L01, D	-
Store, scaled register offset, minus	STR, STRB	2	2	I, L01, D	-
Store, immed pre-indexed	STR, STRB, STRD, STRH	1	2	L01, D, I	-
Store, register pre-indexed, plus, no shift	STR, STRB, STRD, STRH	1	2	L01, D, M0	1
Store, register pre-indexed, minus	STR, STRB, STRD, STRH	2	2	I, L01, D, M0	1
Store, scaled register pre-indexed, plus LSL2	STR, STRB	1	2	L01, D, M0	1
Store, scaled register pre-indexed, other	STR, STRB	2	2	I, L01, D, M0	1
Store, immed post-indexed	STR{T}, STRB{T}, STRD, STRH{T}	1	2	L01, D, I	-
Store, register post-indexed	STRH{T}, STRD	1	2	L01, D, M0	1
Store, register post-indexed	STR{T}, STRB{T}	1	2	L01, D, M0	1
Store, scaled register post-indexed	STR{T}, STRB{T}	1	2	L01, D, M0	2
Store multiple, no writeback	STMIA, STMIB, STMDA, STMDB	N	1/N	L01, D	3
Store multiple, writeback	STMIA, STMIB, STMDA, STMDB, PUSH	N	1/N	L01, D	3

Notes:

1. The address update op goes down pipeline 'I' if the store is unconditional.
2. The address update op goes down pipeline "M" if the store is unconditional.
3. For store multiple instructions, $N = \text{floor}((\text{num_regs} + 3) / 4)$.

3.12 Tag Load Instructions

Table 3-17 AArch64 Tag load instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Load allocation tag	LDG	4	3	L	-
Load multiple allocation tags	LDGM	4	3	L	-

3.13 Tag Store instructions

Table 3-18 AArch64 Tag store instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Store allocation tags to one or two granules, post-index	STG, ST2G	1	2	L01, D, I	-
Store allocation tags to one or two granules, pre-index	STG, ST2G	1	2	L01, D, I	-
Store allocation tags to one or two granules, signed offset	STG, ST2G	1	2	L01, D	-
Store allocation tag to one or two granules, zeroing, post-index	STZG, STZ2G	1	2	L01, D, I	-
Store Allocation Tag to one or two granules, zeroing, pre-index	STZG, STZ2G	1	2	L01, D, I	-
Store allocation tag to two granules, zeroing, signed offset	STZG, STZ2G	1	2	L01, D	-
Store allocation tag and reg pair to memory, post-Index	STGP	1	2	L01, D, I	-
Store allocation tag and reg pair to memory, pre-Index	STGP	1	2	L01, D, I	-
Store allocation tag and reg pair to memory, signed offset	STGP	1	2	L01, D	-
Store multiple allocation tags	STGM	1	2	L01, D	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Store multiple allocation tags, zeroing	STZGM	1	2	L01, D	-

3.14 FP data processing instructions

Table 3-19 AArch64 FP data processing instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
FP absolute value	FABS, FABD	2	2	V	-
FP arithmetic	FADD, FSUB	2	2	V	-
FP compare	FCCMP{E}, FCMP{E}	2	1	V0	-
FP divide, H-form	FDIV	7	2/7	V0	1
FP divide, S-form	FDIV	7 to 10	2/9 to 2/7	V0	1
FP divide, D-form	FDIV	7 to 15	1/7 to 2/7	V0	1
FP min/max	FMIN, FMINNM, FMAX, FMAXNM	2	2	V	-
FP multiply	FMUL, FNMUL	3	2	V	2
FP multiply accumulate	FMADD, FMSUB, FNMADD, FNMSUB	4 (2)	2	V	3
FP negate	FNEG	2	2	V	-
FP round to integral	FRINTA, FRINTI, FRINTM, FRINTN, FRINTP, FRINTX, FRINTZ, FRINT32X, FRINT64X, FRINT32Z, FRINT64Z	3	1	V0	-
FP select	FCSEL	2	2	V	-
FP square root, H-form	FSQRT	7	4/7	V0	1
FP square root, S-form	FSQRT	7 to 9	1/2 to 4/7	V0	1
FP square root, D-form	FSQRT	7 to 16	2/15 to 2/7	V0	1

Table 3-20 AArch32 FP data processing instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
VFP absolute value	VABS	2	2	V	-
VFP arith	VADD, VSUB	2	2	V	-
VFP compare, unconditional	VCMP, VCMPE	2	1	V0	-
VFP compare, conditional	VCMP, VCMPE	4	1	V, V0	-
VFP convert	VCVT{R}, VCVTB, VCVTT, VCVTA, VCVTM, VCVTN, VCVTP	3	1	V0	-
VFP convert to BFloat16	VCVTB, VCVTT	3	1	V0	-
VFP divide, H-form	VDIV	7	4/7	V0	1
VFP divide, S-form	VDIV	7 to 10	4/9 to 4/7	V0	1
VFP divide, D-form	VDIV	7 to 15	1/7 to 2/7	V0	1
VFP max/min	VMAXNM, VMINNM	2	2	V	-
VFP multiply	VMUL, VNMUL	3	2	V	2
VFP multiply accumulate (chained)	VMLA, VMLS, VNMLA, VNMLS	5 (2)	2	V	3
VFP multiply accumulate (fused)	VFMA, VFMS, VFNMA, VFNMS	4 (2)	2	V	3
VFP negate	VNEG	2	2	V	-
VFP round to integral	VRINTA, VRINTM, VRINTN, VRINTP, VRINTR, VRINTX, VRINTZ	3	1	V0	-
VFP select	VSELEQ, VSELGE, VSELGT, VSELVS	2	2	V	-
VFP square root, H-form	VSQRT	7	4/7	V0	1
VFP square root, S-form	VSQRT	7 to 9	1/2 to 4/7	V0	1
VFP square root, D-form	VSQRT	7 to 16	2/15 to 2/7	V0	1

Notes:

1. FP divide and square root operations are performed using an iterative algorithm and block subsequent similar operations to the same pipeline until complete.
2. FP multiply-accumulate pipelines support late forwarding of the result from FP multiply μ OPs to the accumulate operands of an FP multiply-accumulate μ OP. The latter can potentially be issued 1 cycle after the FP multiply μ OP has been issued.
3. FP multiply-accumulate pipelines support late-forwarding of accumulate operands from similar μ OPs, allowing a typical sequence of multiply-accumulate μ OPs to issue one every N cycles (accumulate latency N shown in parentheses).

3.15 FP miscellaneous instructions

Table 3-21 AArch64 FP miscellaneous instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
FP convert, from gen to vec reg	SCVTF, UCVTF	3	1	M0	-
FP convert, from vec to gen reg	FCVTAS, FCVTAU, FCVTMS, FCVTMU, FCVTNS, FCVTNU, FCVTPS, FCVTPU, FCVTZS, FCVTZU	3	1	V	-
FP convert, Javascript from vec to gen reg	FJCVTZS	3	1	V0	-
FP convert, from vec to vec reg	FCVT, FCVTXN	3	1	V0	-
FP move, immed	FMOV	2	2	V	-
FP move, register	FMOV	2	2	V	-
FP transfer, from gen to low half of vec reg	FMOV	3	1	M0	-
FP transfer, from gen to high half of vec reg	FMOV	5	1	M0, V	-
FP transfer, from vec to gen reg	FMOV	2	1	V	-

Table 3-22 AArch32 FP miscellaneous instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
VFP move, extraction	VMOVX	2	2	V	-
VFP move, immed	VMOV	2	2	V	-
VFP move, insert	VINS	2	2	V	-
VFP move, register	VMOV	2	2	V	-
VFP transfer, core to vfp, single reg to S-reg, cond	VMOV	5	1	M0, V	-
VFP transfer, core to vfp, single reg to S-reg, uncond	VMOV	3	1	M0	-
VFP transfer, core to vfp, single reg to upper/lower half of D-reg	VMOV	5	1	M0, V	-

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
VFP transfer, core to vfp, 2 regs to 2 S-reg, cond	VMOV	6	1/2	M0, V	-
VFP transfer, core to vfp, 2 regs to 2 S-reg, uncond	VMOV	4	1/2	M0	-
VFP transfer, core to vfp, 2 regs to D-reg, cond	VMOV	5	1	M0, V	-
VFP transfer, core to vfp, 2 regs to D-reg, uncond	VMOV	3	1	M0	-
VFP transfer, vfp S-reg or upper/lower half of vfp D-reg to core reg, cond	VMOV	3	1	V, I	-
VFP transfer, vfp S-reg or upper/lower half of vfp D-reg to core reg, uncond	VMOV	2	1	V	-
VFP transfer, vfp 2 S-reg or D-reg to 2 core regs, cond	VMOV	3	1	V, I	-
VFP transfer, vfp 2 S-reg or D-reg to 2 core regs, uncond	VMOV	2	1	V	-

3.16 FP load instructions

The latencies shown assume the memory access hits in the Level 1 Data Cache and represent the maximum latency to load all the vector registers written by the instruction. Compared to standard loads, an extra cycle is required to forward results to FP/ASIMD pipelines.

Table 3-23 AArch64 FP load instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Load vector reg, literal, S/D/Q forms	LDR	6	3	L	-
Load vector reg, unscaled immed	LDUR	6	3	L	-
Load vector reg, immed post-index	LDR	6	3	L, I	-
Load vector reg, immed pre-index	LDR	6	3	L, I	-
Load vector reg, unsigned immed	LDR	6	3	L	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Load vector reg, register offset, basic	LDR	6	3	L	-
Load vector reg, register offset, scale, S/D-form	LDR	6	3	L	-
Load vector reg, register offset, scale, H/Q-form	LDR	7	3	I, L	-
Load vector reg, register offset, extend	LDR	6	3	L	-
Load vector reg, register offset, extend, scale, S/D-form	LDR	6	3	L	-
Load vector reg, register offset, extend, scale, H/Q-form	LDR	7	3	I, L	-
Load vector pair, immed offset, S/D-form	LDP, LDNP	6	3	L	-
Load vector pair, immed offset, Q-form	LDP, LDNP	6	3/2	L	-
Load vector pair, immed post-index, S/D-form	LDP	6	3	I, L	-
Load vector pair, immed post-index, Q-form	LDP	6	3/2	L, I	-
Load vector pair, immed pre-index, S/D-form	LDP	6	3	I, L	-
Load vector pair, immed pre-index, Q-form	LDP	6	3/2	L, I	-

Table 3-24 AArch32 FP load instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
FP load, register	VLDR	6	3 (2)	L	1, 6, 7
FP load multiple, S form	VLDmia, VLDMia, VPOp	N(N*)	3/R (2/R)	L	1, 2, 3, 4, 6, 7
FP load multiple, D form	VLDmia, VLDMia, VPOp	N(N*)	3/R (2/R)	L, V	1, 2, 3, 4, 6, 7
(FP load, writeback forms)	-	(1)	-	+ I	5, 7

Notes:

Condition loads have an extra uop which goes down pipeline V and have 2 cycle extra latency compared to their unconditional counterparts.

1. N is (num_reg)/6 + 5.

2. N^* is $(\text{num_reg})/4 + 5$.
3. R is $\text{num_reg}/2$.
4. Writeback forms of load instructions require an extra μOP to update the base address. This update is typically performed in parallel with or prior to the load μOP (update latency shown in parentheses).
5. The number in parenthesis represents the latency and throughput of conditional loads.
6. Conditional loads go down the L01 pipe.

3.17 FP store instructions

Stores MOPs are split into store address and store data μOPs . Once executed, stores are buffered and committed in the background.

Table 3-25 AArch64 FP store instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Store vector reg, unscaled immed, B/H/S/D-form	STUR	2	2	L01, V	-
Store vector reg, unscaled immed, Q-form	STUR	2	2	L01, V	-
Store vector reg, immed post-index, B/H/S/D-form	STR	2	2	L01, V, I	-
Store vector reg, immed post-index, Q-form	STR	2	2	L01, V, I	-
Store vector reg, immed pre-index, B/H/S/D-form	STR	2	2	L01, V, I	-
Store vector reg, immed pre-index, Q-form	STR	2	2	L01, V, I	-
Store vector reg, unsigned immed, B/H/S/D-form	STR	2	2	L01, V	-
Store vector reg, unsigned immed, Q-form	STR	2	2	L01, V	-
Store vector reg, register offset, basic, B/H/S/D-form	STR	2	2	L01, V	-
Store vector reg, register offset, basic, Q-form	STR	2	2	L01, V	-
Store vector reg, register offset, scale, H-form	STR	2	2	I, L01, V	-
Store vector reg, register offset, scale, S/D-form	STR	2	2	L01, V	-
Store vector reg, register offset, scale, Q-form	STR	2	2	I, L01, V	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Store vector reg, register offset, extend, B/H/S/D-form	STR	2	2	L01, V	-
Store vector reg, register offset, extend, Q-form	STR	2	2	L01, V	-
Store vector reg, register offset, extend, scale, H-form	STR	2	2	I, L01, V	-
Store vector reg, register offset, extend, scale, S/D-form	STR	2	2	L01, V	-
Store vector reg, register offset, extend, scale, Q-form	STR	2	2	I, L01, V	-
Store vector pair, immed offset, S-form	STP, STNP	2	2	L01, V	-
Store vector pair, immed offset, D-form	STP, STNP	2	2	L01, V	-
Store vector pair, immed offset, Q-form	STP, STNP	2	1	L01, V	-
Store vector pair, immed post-index, S-form	STP	2	2	I, L01, V	-
Store vector pair, immed post-index, D-form	STP	2	2	I, L01, V	-
Store vector pair, immed post-index, Q-form	STP	2	1	I, L01, V	-
Store vector pair, immed pre-index, S-form	STP	2	2	I, L01, V	-
Store vector pair, immed pre-index, D-form	STP	2	2	I, L01, V	-
Store vector pair, immed pre-index, Q-form	STP	2	1	I, L01, V	-

Table 3-26 AArch32 FP store instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
FP store, immed offset	VSTR	2	2	L01, V	-
FP store multiple, S-form	VSTMIA, VSTMDB, VPUSH	N + 1	2/R	L01, V	1, 2
FP store multiple, D-form	VSTMIA, VSTMDB, VPUSH	N + 1	2/R	L01, V	1, 2
(FP store, writeback forms)	-	(1)	-	+ I	3

Notes:

1. For store multiple instructions, $N = (\text{num_reg}/2)$
2. R is num_regs.
3. Writeback forms of store instructions require an extra μOP to update the base address. This update is typically performed in parallel with or prior to the store μOP (update latency shown in parentheses).

3.18 ASIMD integer instructions

Table 3-27 AArch64 ASIMD integer instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD absolute diff	SABD, UABD	2	2	V	-
ASIMD absolute diff accum	SABA, UABA	4(1)	1	V1	2
ASIMD absolute diff accum long	SABAL(2), UABAL(2)	4(1)	1	V1	2
ASIMD absolute diff long	SABDL(2), UABDL(2)	2	2	V	-
ASIMD arith, basic	ABS, ADD, NEG, SADDL(2), SADDW(2), SHADD, SHSUB, SSUBL(2), SSUBW(2), SUB, UADDL(2), UADDW(2), UHADD, UHSUB, USUBL(2), USUBW(2)	2	2	V	-
ASIMD arith, complex	ADDHN(2), RADDHN(2), RSUBHN(2), SQABS, SQADD, SQNEG, SQSUB, SRHADD, SUBHN(2), SUQADD, UQADD, UQSUB, URHADD, USQADD	2	2	V	-
ASIMD arith, pair-wise	ADDP, SADDLP, UADDLP	2	2	V	-
ASIMD arith, reduce, 4H/4S	ADDV, SADDLV, UADDLV	2	1	V1	-
ASIMD arith, reduce, 8B/8H	ADDV, SADDLV, UADDLV	4	1	V1, V	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD arith, reduce, 16B	ADDV, SADDLV, UADDLV	4	1	V1	-
ASIMD compare	CMEQ, CMGE, CMGT, CMHI, CMHS, CMLE, CMLT, CMTST	2	2	V	-
ASIMD dot product	SDOT, UDOT	3 (1)	2	V	2
ASIMD dot product using signed and unsigned integers	SUDOT, USDOT	3(1)	2	V	2
ASIMD logical	AND, BIC, EOR, MOV, MVN, NOT, ORN, ORR	2	2	V	-
ASIMD matrix multiply-accumulate	SMMLA, UMMLA, USMMLA	3(1)	2	V	2
ASIMD max/min, basic and pair-wise	SMAX, SMAXP, SMIN, SMINP, UMAX, UMAXP, UMIN, UMINP	2	2	V	-
ASIMD max/min, reduce, 4H/4S	SMAXV, SMINV, UMAXV, UMINV	2	2	V1	-
ASIMD max/min, reduce, 8B/8H	SMAXV, SMINV, UMAXV, UMINV	4	1	V1, V	-
ASIMD max/min, reduce, 16B	SMAXV, SMINV, UMAXV, UMINV	4	½	V1	-
ASIMD multiply	MUL, SQDMULH, SQRDMULH	4	1	V0	-
ASIMD multiply accumulate	MLA, MLS	4(1)	1	V0	1
ASIMD multiply accumulate high	SQRDMLAH, SQRDMLSH	4(2)	1	V0	1
ASIMD multiply accumulate long	SMLAL(2), SMLSL(2), UMLAL(2), UMLSL(2)	4(1)	1	V0	1
ASIMD multiply accumulate saturating long	SQDMLAL(2), SQDMLSL(2)	4(2)	1	V0	1
ASIMD multiply/multiply long (8x8) polynomial, D-form	PMUL, PMULL(2)	3	1	V0	3
ASIMD multiply/multiply long (8x8) polynomial, Q-form	PMUL, PMULL(2)	3	1	V0	3
ASIMD multiply long	SMULL(2), UMULL(2), SQDMULL(2)	3	2	V	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD pairwise add and accumulate long	SADALP, UADALP	4(1)	1	V1	2
ASIMD shift accumulate	SSRA, SRSRA, USRA, URSRA	4(1)	1	V1	2
ASIMD shift by immed, basic	SHL, SHLL(2), SHRN(2), SSHLL(2), SSHR, SXTL(2), USHLL(2), USHR, UXTL(2)	2	1	V1	-
ASIMD shift by immed and insert, basic	SLI, SRI	2	1	V1	-
ASIMD shift by immed, complex	RSHRN(2), SQRSHRN(2), SQRSHRUN(2), SQSHL{U}, SQSHRN(2), SQSHRUN(2), SRSHR, UQRSHRN(2), UQSHL, UQSHRN(2), URSRHR	4	1	V1	-
ASIMD shift by register, basic	SSHL, USHL	2	1	V1	-
ASIMD shift by register, complex	SRSHL, SQRSHL, SQSHL, URSHL, UQRSHL, UQSHL	4	1	V1	-

Table 3-28 AArch32 ASIMD integer instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD absolute diff	VABD	2	2	V	-
ASIMD absolute diff accum	VABA	4(1)	1	V1	2
ASIMD absolute diff accum long	VABAL	4(1)	1	V1	2
ASIMD absolute diff long	VABDL	2	2	V	-
ASIMD arith, basic	VADD, VADDL, VADDW, VNEG, VSUB, VSUBL, VSUBW	2	2	V	-

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD arith, complex	VABS, VADDHN, VHADD, VHSUB, VQABS, VQADD, VQNEG, VQSUB, VRADDHN, VRHADD, VRSUBHN, VSUBHN	2	2	V	-
ASIMD arith, pair-wise	VPADD, VPADDL	2	2	V	-
ASIMD compare	VCEQ, VCGE, VCGT, VCLE, VTST	2	2	V	-
ASIMD dot product	VSDOT, VUDOT	3(1)	2	V	2
ASIMD dot product using signed and unsigned integers	VSUDOT, VUSDOT	3(1)	2	V	2
ASIMD logical	VAND, VBIC, VMVN, VORR, VORN, VEOR	2	2	V	-
ASIMD matrix multiply-accumulate	VSMMLA, VUMMLA, VUSMMLA	3(1)	2	V	2
ASIMD max/min	VMAX, VMIN, VPMAX, VPMIN	2	2	V	-
ASIMD multiply	VMUL, VQDMULH, VQRDMULH	4	1	V0	-
ASIMD multiply accumulate	VMLA, VMLS	4(1)	1	V0	1
ASIMD multiply accumulate long	VMLAL, VMLSL	4(1)	1	V0	1
ASIMD multiply accumulate saturating long	VQDMLAL, VQDMLSL	4	1	V0	-
ASIMD multiply/multiply long (8x8) polynomial, D-form	VMUL (.P8), VMULL (.P8)	3	1	V0	-
ASIMD multiply (8x8) polynomial, Q-form	VMUL (.P8)	3	1	V0	-
ASIMD multiply long	VMULL (.S, .I), VQDMULL	3	1	V0	-
ASIMD pairwise add and accumulate	VPADAL	4(1)	1	V1	1
ASIMD shift accumulate	VSRA, VRSRA	4(1)	1	V1	1
ASIMD shift by immed, basic	VMOVL, VSHL, VSHLL, VSHR, VSHRN	2	1	V1	-

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD shift by immed and insert, basic	VSLI, VSRI	2	1	V1	-
ASIMD shift by immed, complex	VQRSHRN, VQRSHRUN, VQSHL{U}, VQSHRN, VQSHRUN, VRSHR, VRSHRN	4	1	V1	-
ASIMD shift by register, basic	VSHL	2	1	V1	-
ASIMD shift by register, complex	VQRSHL, VQSHL, VRSHL	4	1	V1	-

Notes:

1. Multiply-accumulate pipelines support late-forwarding of accumulate operands from similar μ OPs, allowing a typical sequence of integer multiply-accumulate μ OPs to issue one every cycle or one every other cycle (accumulate latency shown in parentheses).
2. Other accumulate pipelines also support late-forwarding of accumulate operands from similar μ OPs, allowing a typical sequence of such μ OPs to issue one every cycle (accumulate latency shown in parentheses).
3. This category includes instructions of the form “PMULL Vd.8H, Vn.8B, Vm.8B” and “PMULL2 Vd.8H, Vn.16B, Vm.16B”.

3.19 ASIMD floating-point instructions

Table 3-29 AArch64 ASIMD floating-point instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD FP absolute value/difference	FABS, FABD	2	2	V	-
ASIMD FP arith, normal	FADD, FSUB, FADDP	2	2	V	-
ASIMD FP compare	FACGE, FACGT, FCMEQ, FCMGE, FCMGT, FCMLE, FCMLT	2	2	V	-
ASIMD FP complex add	FCADD	2	2	V	-
ASIMD FP complex multiply add	FCMLA	4(2)	2	V	1
ASIMD FP convert, long (F16 to F32)	FCVTL(2)	4	1/2	V0	-
ASIMD FP convert, long (F32 to F64)	FCVTL(2)	3	1	V0	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD FP convert, narrow (F32 to F16)	FCVTN(2)	4	1/2	V0	-
ASIMD FP convert, narrow (F64 to F32)	FCVTN(2), FCVTXN(2)	3	2	V0	-
ASIMD FP convert, other, D-form F32 and Q-form F64	FCVTAS, FCVTAU, FCVTMS, FCVTMU, FCVTNS, FCVTNU, FCVTPS, FCVTPU, FCVTZS, FCVTZU, SCVTF, UCVTF	3	1	V0	-
ASIMD FP convert, other, D-form F16 and Q-form F32	FCVTAS, VCVTAU, FCVTMS, FCVTMU, FCVTNS, FCVTNU, FCVTPS, FCVTPU, FCVTZS, FCVTZU, SCVTF, UCVTF	4	1/2	V0	-
ASIMD FP convert, other, Q-form F16	FCVTAS, VCVTAU, FCVTMS, FCVTMU, FCVTNS, FCVTNU, FCVTPS, FCVTPU, FCVTZS, FCVTZU, SCVTF, UCVTF	6	1/4	V0	-
ASIMD FP divide, D-form, F16	FDIV	7	1/7	V0	3
ASIMD FP divide, D-form, F32	FDIV	7 to 10	2/9 to 2/7	V0	3
ASIMD FP divide, Q-form, F16	FDIV	10 to 13	1/13 to 1/10	V0	3
ASIMD FP divide, Q-form, F32	FDIV	7 to 10	1/9 to 1/7	V0	3
ASIMD FP divide, Q-form, F64	FDIV	7 to 15	1/14 to 1/7	V0	3
ASIMD FP max/min, normal	FMAX, FMAXNM, FMIN, FMINNM	2	2	V	-
ASIMD FP max/min, pairwise	FMAXP, FMAXNMP, FMINP, FMINNMP	2	2	V	-
ASIMD FP max/min, reduce, F32 and D-form F16	FMAXV, FMAXNMV, FMINV, FMINNMV	4	1	V	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD FP max/min, reduce, Q-form F16	FMAXV, FMAXNMV, FMINV, FMINNMV	6	2/3	V	-
ASIMD FP multiply	FMUL, FMULX	3	2	V	2
ASIMD FP multiply accumulate	FMLA, FMLS	4(2)	2	V	1
ASIMD FP multiply accumulate long	FMLAL(2), FMLSL(2)	5(2)	2	V	1
ASIMD FP negate	FNEG	2	2	V	-
ASIMD FP round, D-form F32 and Q-form F64	FRINTA, FRINTI, FRINTM, FRINTN, FRINTP, FRINTX, FRINTZ, FRINT32X, FRINT64X, FRINT32Z, FRINT64Z	3	1	V0	-
ASIMD FP round, D-form F16 and Q-form F32	FRINTA, FRINTI, FRINTM, FRINTN, FRINTP, FRINTX, FRINTZ, FRINT32X, FRINT64X, FRINT32Z, FRINT64Z	4	1/2	V0	-
ASIMD FP round, Q-form F16	FRINTA, FRINTI, FRINTM, FRINTN, FRINTP, FRINTX, FRINTZ, FRINT32X, FRINT64X, FRINT32Z, FRINT64Z	6	1/4	V0	-
ASIMD FP square root, D-form, F16	FSQRT	7	1/7	V0	3
ASIMD FP square root, D-form, F32	FSQRT	7 to 10	2/9 to 2/7	V0	3
ASIMD FP square root, Q-form, F16	FSQRT	11 to 13	1/13 to 1/11	V0	3
ASIMD FP square root, Q-form, F32	FSQRT	7 to 10	1/9 to 1/7	V0	3
ASIMD FP square root, Q-form, F64	FSQRT	7 to 16	1/15 to 1/7	V0	3

Table 3-30 AArch32 ASIMD floating-point instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD FP absolute value	VABS	2	2	V	-
ASIMD FP arith	VABD, VADD, VPADD, VSUB	2	2	V	-
ASIMD FP compare	VACGE, VACGT, VACLE, VACLT, VCEQ, VCGE, VCGT, VCLE	2	2	V	-
ASIMD FP complex add	VCADD	2	2	V	-
ASIMD FP complex multiply add	VCMLA	4(2)	2	V	2
ASIMD FP convert, integer, D-form	VCVT, VCVTA, VCVTM, VCVTN, VCVTP	3	1	V0	-
ASIMD FP convert, integer, Q-form	VCVT, VCVTA, VCVTM, VCVTN, VCVTP	4	1/2	V0	-
ASIMD FP convert, fixed, D-form	VCVT	3	1	V0	-
ASIMD FP convert, fixed, Q-form	VCVT	4	1/2	V0	-
ASIMD FP convert, half-precision	VCVT	4	1/2	V0	-
ASIMD FP max/min	VMAX, VMIN, VPMAX, VPMIN, VMAXNM, VMINNM	2	2	V	-
ASIMD FP multiply	VMUL, VNMUL	3	2	V	2
ASIMD FP chained multiply accumulate	VMLA, VMLS	5(2)	2	V	1
ASIMD FP fused multiply accumulate	VFMA, VFMS	4(2)	2	V	1
ASIMD FP multiply accumulate long	VFMAL, VFMSL	5(2)	2	V	1
ASIMD FP negate	VNEG	2	2	V	
ASIMD FP round to integral, D-form	VRINTA, VRINTM, VRINTN, VRINTP, VRINTX, VRINTZ	3	1/2	V0	-

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD FP round to integral, Q-form	VRINTA, VRINTM, VRINTN, VRINTP, VRINTX, VRINTZ	4	1	V0	-

Notes:

1. ASIMD multiply-accumulate pipelines support late-forwarding of accumulate operands from similar μ OPs, allowing a typical sequence of floating-point multiply-accumulate μ OPs to issue one every N cycles (accumulate latency N shown in parentheses).
2. ASIMD multiply-accumulate pipelines support late forwarding of the result from ASIMD FP multiply μ OPs to the accumulate operands of an ASIMD FP multiply-accumulate μ OP. The latter can potentially be issued 1 cycle after the ASIMD FP multiply μ OP has been issued.
3. ASIMD divide and square root operations are performed using an iterative algorithm and block subsequent similar operations to the same pipeline until complete.

3.20 ASIMD BFloat16 (BF16) instructions

Table 3-31 AArch64 ASIMD BFloat (BF16) instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD convert, F32 to BF16	BFCVTN, BFCVTN2	4	1	V0	-
ASIMD dot product	BFDOT	4(2)	2	V	1
ASIMD matrix multiply accumulate	BFMMLA	5(3)	2	V	1
ASIMD multiply accumulate long	BFMLALB, BFMLALT	4(2)	2	V	1
Scalar convert, F32 to BF16	BFCVT	3	1	V0	-

Table 3-32 AArch32 ASIMD BFloat (BF16) instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD convert, F32 to BF16	VCVTB, VCVTT	4	1	V0	-
ASIMD dot product	VDOT	4(2)	2	V	1
ASIMD matrix multiply accumulate	VMMLA	5(3)	2	V	1
ASIMD multiply accumulate long	VFMAB, VFMAT	4(2)	2	V	1
Scalar convert, F32 to BF16	VCVT	3	1	V0	-

Notes:

1. ASIMD pipelines that execute these instructions support late-forwarding of accumulate operands from similar μ OPs, allowing a typical sequence of μ OPs to issue one every N cycles (accumulate latency N shown in parentheses).

3.21 ASIMD miscellaneous instructions

Table 3-33 AArch64 ASIMD miscellaneous instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD bit reverse	RBIT	2	2	V	-
ASIMD bitwise insert	BIF, BIT, BSL	2	2	V	-
ASIMD count	CLS, CLZ, CNT	2	2	V	-
ASIMD duplicate, gen reg	DUP	3	1	M0	-
ASIMD duplicate, element	DUP	2	2	V	-
ASIMD extract	EXT	2	2	V	-
ASIMD extract narrow	XTN(2)	2	2	V	-
ASIMD extract narrow, saturating	SQXTN(2), SQXTUN(2), UQXTN(2)	4	1	V1	-
ASIMD insert, element to element	INS	2	2	V	-
ASIMD move, FP immed	FMOV	2	2	V	-
ASIMD move, integer immed	MOVI, MVNI	2	2	V	-
ASIMD reciprocal and square root estimate, D-form U32	URECPE, URSQRTE	3	1	V0	-
ASIMD reciprocal and square root estimate, Q-form U32	URECPE, URSQRTE	4	1/2	V0	-
ASIMD reciprocal and square root estimate, D-form F32 and scalar forms	FRECPE, FRSQRTE	3	1	V0	-
ASIMD reciprocal and square root estimate, D-form F16 and Q-form F32	FRECPE, FRSQRTE	4	1/2	V0	-
ASIMD reciprocal and square root estimate, Q-form F16	FRECPE, FRSQRTE	6	1/4	V0	-
ASIMD reciprocal exponent	FRECPX	3	1	V0	-
ASIMD reciprocal step	FRECPS, FRSQRTS	4	2	V	-
ASIMD reverse	REV16, REV32, REV64	2	2	V	-
ASIMD table lookup, 1 or 2 table regs	TBL	2	2	V	-
ASIMD table lookup, 3 table regs	TBL	4	1	V	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD table lookup, 4 table regs	TBL	4	2/3	V	-
ASIMD table lookup extension, 1 table reg	TBX	2	2	V	-
ASIMD table lookup extension, 2 table reg	TBX	4	1	V	-
ASIMD table lookup extension, 3 table reg	TBX	6	2/3	V	-
ASIMD table lookup extension, 4 table reg	TBX	6	2/5	V	-
ASIMD transfer, element to gen reg	UMOV, SMOV	2	1	V	-
ASIMD transfer, gen reg to element	INS	5	1	M0, V	-
ASIMD transpose	TRN1, TRN2	2	2	V	-
ASIMD unzip/zip	UZP1, UZP2, ZIP1, ZIP2	2	2	V	-

Table 3-34 AArch32 ASIMD miscellaneous instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD bitwise insert	VBIF, VBIT, VBSL	2	2	V	-
ASIMD count	VCLS, VCLZ, VCNT	2	2	V	-
ASIMD duplicate, core reg	VDUP	3	1	M0	-
ASIMD duplicate, scalar	VDUP	2	2	V	-
ASIMD extract	VEXT	2	2	V	-
ASIMD move, immed	VMOV	2	2	V	-
ASIMD move, register	VMOV	2	2	V	-
ASIMD move, narrowing	VMOVN	2	2	V	-
ASIMD move, saturating	VQMOVN, VQMOVUN	4	1	V1	-
ASIMD reciprocal estimate, D-form F32 and F64	VRECPE, VRSQRTE	3	1	V0	-
ASIMD reciprocal estimate, D-form F16 and Q-form F32	VRECPE, VRSQRTE	4	1/2	V0	

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD reciprocal estimate, Q-form F16	VRECPE, VRSQRTE	6	1/4	V0	-
ASIMD reciprocal step	VRECPS, VRSQRSTS	5	2	V	-
ASIMD reverse	VREV16, VREV32, VREV64	2	2	V	-
ASIMD swap	VSWP	4	2/3	V	-
ASIMD table lookup, 1 or 2 table regs	VTBL	2	2	V	-
ASIMD table lookup, 3 table regs	VTBL	4	1	V	-
ASIMD table lookup, 4 table regs	VTBL	6	2/3	V	-
ASIMD table lookup extension, 1 reg	VTBX	2	2	V	-
ASIMD table lookup extension, 2 table reg	VTBX	4	1	V	-
ASIMD table lookup extension, 3 table reg	VTBX	6	2/3	V	-
ASIMD table lookup extension, 4 table reg	VTBX	6	2/5	V	-
ASIMD transfer, scalar to core reg, word	VMOV	2	1	V	-
ASIMD transfer, scalar to core reg, byte/hword	VMOV	3	1	V, I	-
ASIMD transfer, core reg to scalar	VMOV	5	1	M0, V	-
ASIMD transpose	VTRN	4	2/3	V	-
ASIMD unzip/zip	VUZP, VZIP	4	2/3	V	-

3.22 ASIMD load instructions

The latencies shown assume the memory access hits in the Level 1 Data Cache and represent the maximum latency to load all the vector registers written by the instruction. Compared to standard loads, an extra cycle is required to forward results to FP/ASIMD pipelines.

Table 3-35 AArch64 ASIMD load instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD load, 1 element, multiple, 1 reg, D-form	LD1	6	3	L	-
ASIMD load, 1 element, multiple, 1 reg, Q-form	LD1	6	3	L	-
ASIMD load, 1 element, multiple, 2 reg, D-form	LD1	6	3/2	L	-
ASIMD load, 1 element, multiple, 2 reg, Q-form	LD1	6	3/2	L	-
ASIMD load, 1 element, multiple, 3 reg, D-form	LD1	6	1	L	-
ASIMD load, 1 element, multiple, 3 reg, Q-form	LD1	6	1	L	-
ASIMD load, 1 element, multiple, 4 reg, D-form	LD1	7	3/4	L	-
ASIMD load, 1 element, multiple, 4 reg, Q-form	LD1	7	3/4	L	-
ASIMD load, 1 element, one lane, B/H/S	LD1	8	2	L, V	-
ASIMD load, 1 element, one lane, D	LD1	8	2	L, V	-
ASIMD load, 1 element, all lanes, D-form, B/H/S	LD1R	8	2	L, V	-
ASIMD load, 1 element, all lanes, D-form, D	LD1R	8	2	L, V	-
ASIMD load, 1 element, all lanes, Q-form	LD1R	8	2	L, V	-
ASIMD load, 2 element, multiple, D-form, B/H/S	LD2	8	2	L, V	-
ASIMD load, 2 element, multiple, Q-form, B/H/S	LD2	8	3/2	L, V	-
ASIMD load, 2 element, multiple, Q-form, D	LD2	8	3/2	L, V	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD load, 2 element, one lane, B/H	LD2	8	2	L, V	-
ASIMD load, 2 element, one lane, S	LD2	8	2	L, V	-
ASIMD load, 2 element, one lane, D	LD2	8	2	L, V	-
ASIMD load, 2 element, all lanes, D-form, B/H/S	LD2R	8	2	L, V	-
ASIMD load, 2 element, all lanes, D-form, D	LD2R	8	2	L, V	-
ASIMD load, 2 element, all lanes, Q-form	LD2R	8	2	L, V	-
ASIMD load, 3 element, multiple, D-form, B/H/S	LD3	8	2/3	L, V	-
ASIMD load, 3 element, multiple, Q-form, B/H/S	LD3	8	2/3	L, V	-
ASIMD load, 3 element, multiple, Q-form, D	LD3	8	2/3	L, V	-
ASIMD load, 3 element, one lane, B/H	LD3	8	2/3	L, V	-
ASIMD load, 3 element, one lane, S	LD3	8	2/3	L, V	-
ASIMD load, 3 element, one lane, D	LD3	8	2/3	L, V	-
ASIMD load, 3 element, all lanes, D-form, B/H/S	LD3R	8	2/3	L, V	-
ASIMD load, 3 element, all lanes, D-form, D	LD3R	8	2/3	L, V	-
ASIMD load, 3 element, all lanes, Q-form, B/H/S	LD3R	8	2/3	L, V	-
ASIMD load, 3 element, all lanes, Q-form, D	LD3R	8	2/3	L, V	-
ASIMD load, 4 element, multiple, D-form, B/H/S	LD4	8	1	L, V	-
ASIMD load, 4 element, multiple, Q-form, B/H/S	LD4	9	1/2	L, V	-
ASIMD load, 4 element, multiple, Q-form, D	LD4	9	1/2	L, V	-
ASIMD load, 4 element, one lane, B/H	LD4	8	1	L, V	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD load, 4 element, one lane, S	LD4	8	1	L, V	-
ASIMD load, 4 element, one lane, D	LD4	8	1	L, V	-
ASIMD load, 4 element, all lanes, D-form, B/H/S	LD4R	8	1	L, V	-
ASIMD load, 4 element, all lanes, D-form, D	LD4R	8	1	L, V	-
ASIMD load, 4 element, all lanes, Q-form, B/H/S	LD4R	8	1	L, V	-
ASIMD load, 4 element, all lanes, Q-form, D	LD4R	8	1	L, V	-
(ASIMD load, writeback form)	-	-	-	I	1

Table 3-36 AArch32 ASIMD load instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD load, 1 element, multiple, 1 reg	VLD1	5	3(2)	L	2
ASIMD load, 1 element, multiple, 2 reg	VLD1	5	3(2)	L	2
ASIMD load, 1 element, multiple, 3 reg	VLD1	5	3/2(1)	L	2
ASIMD load, 1 element, multiple, 4 reg	VLD1	5	3/2(1)	L	2
ASIMD load, 1 element, one lane	VLD1	7	3(2)	L, V	2
ASIMD load, 1 element, all lanes, 1 reg	VLD1	7	3(2)	L, V	2
ASIMD load, 1 element, all lanes, 2 reg	VLD1	7	1	L, V	2
ASIMD load, 2 element, multiple, 2 reg	VLD2	7	1	L, V	2
ASIMD load, 2 element, multiple, 4 reg	VLD2	8	1/2	L, V	2
ASIMD load, 2 element, one lane, size 32	VLD2	7	1	L, V	2

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD load, 2 element, one lane, size 8/16	VLD2	7	1	L, V	2
ASIMD load, 2 element, all lanes	VLD2	7	1	L, V	2
ASIMD load, 3 element, multiple, 3 reg	VLD3	8	2/3 (1)	L, V	2
ASIMD load, 3 element, one lane, size 32	VLD3	8	2/3 (1)	L, V	2
ASIMD load, 3 element, one lane, size 8/16	VLD3	8	2/3 (1)	L, V	2
ASIMD load, 3 element, all lanes	VLD3	8	2/3 (1)	L, V	2
ASIMD load, 4 element, multiple, 4 reg	VLD4	8	1/2	L, V	2
ASIMD load, 4 element, one lane, size 32	VLD4	8	1/2	L, V	2
ASIMD load, 4 element, one lane, size 8/16	VLD4	8	1/2	L, V	2
ASIMD load, 4 element, all lanes	VLD4	8	1/2	L, V	2
(ASIMD load, writeback form)	-	-	-	I	1

Notes:

1. Writeback forms of load instructions require an extra μ OP to update the base address. This update is typically performed in parallel with the load μ OP (update latency shown in parentheses).
2. Conditional loads go down L01 pipe and the number in parenthesis represents their throughput when different from the unconditional forms.

3.23 ASIMD store instructions

Stores MOPs are split into store address and store data μ OPs. Once executed, stores are buffered and committed in the background.

Table 3-37 AArch64 ASIMD store instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD store, 1 element, multiple, 1 reg, D-form	ST1	2	2	L01, V	-
ASIMD store, 1 element, multiple, 1 reg, Q-form	ST1	2	2	L01, V	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD store, 1 element, multiple, 2 reg, D-form	ST1	2	2	L01, V	-
ASIMD store, 1 element, multiple, 2 reg, Q-form	ST1	2	1	L01, V	-
ASIMD store, 1 element, multiple, 3 reg, D-form	ST1	2	1	L01, V	-
ASIMD store, 1 element, multiple, 3 reg, Q-form	ST1	2	2/3	L01, V	-
ASIMD store, 1 element, multiple, 4 reg, D-form	ST1	2	1	L01, V	-
ASIMD store, 1 element, multiple, 4 reg, Q-form	ST1	2	1/2	L01, V	-
ASIMD store, 1 element, one lane, B/H/S	ST1	4	1	L01, V	-
ASIMD store, 1 element, one lane, D	ST1	4	1	L01, V	-
ASIMD store, 2 element, multiple, D-form, B/H/S	ST2	4	1	V, L01	-
ASIMD store, 2 element, multiple, Q-form, B/H/S	ST2	4	1/2	V, L01	-
ASIMD store, 2 element, multiple, Q-form, D	ST2	4	1/2	V, L01	-
ASIMD store, 2 element, one lane, B/H/S	ST2	4	1	V, L01	-
ASIMD store, 2 element, one lane, D	ST2	4	1	V, L01	-
ASIMD store, 3 element, multiple, D-form, B/H/S	ST3	5	1/2	V, L01	-
ASIMD store, 3 element, multiple, Q-form, B/H/S	ST3	6	1/3	V, L01	-
ASIMD store, 3 element, multiple, Q-form, D	ST3	6	1/3	V, L01	-
ASIMD store, 3 element, one lane, B/H	ST3	5	1/2	V, L01	-
ASIMD store, 3 element, one lane, S	ST3	5	1/2	V, L01	-
ASIMD store, 3 element, one lane, D	ST3	5	1/2	V, L01	-
ASIMD store, 4 element, multiple, D-form, B/H/S	ST4	6	1/3	V, L01	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD store, 4 element, multiple, Q-form, B/H/S	ST4	7	1/6	V, L01	-
ASIMD store, 4 element, multiple, Q-form, D	ST4	5	1/4	V, L01	-
ASIMD store, 4 element, one lane, B/H/S	ST4	6	2/3	V, L01	-
ASIMD store, 4 element, one lane, D	ST4	4	1/2	V, L01	-
(ASIMD store, writeback form)	-	-	-	I	1

Table 3-38 AArch32 ASIMD store instructions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD store, 1 element, multiple, 1 reg	VST1	2	2	L01, V	-
ASIMD store, 1 element, multiple, 2 reg	VST1	2	2	L01, V	-
ASIMD store, 1 element, multiple, 3 reg	VST1	2	1	L01, V	-
ASIMD store, 1 element, multiple, 4 reg	VST1	2	1	L01, V	-
ASIMD store, 1 element, one lane	VST1	4	1	V, L01	-
ASIMD store, 2 element, multiple, 2 reg	VST2	5	2/3	V, L01	-
ASIMD store, 2 element, multiple, 4 reg	VST2	5	1/3	V, L01	-
ASIMD store, 2 element, one lane	VST2	4	1	V, L01	-
ASIMD store, 3 element, multiple, 3 reg	VST3	5	1/2	V, L01	-
ASIMD store, 3 element, one lane, size 32	VST3	4	1/2	V, L01	-
ASIMD store, 3 element, one lane, size 8/16	VST3	4	1/2	V, L01	-
ASIMD store, 4 element, multiple, 4 reg	VST4	5	1/3	V, L01	-

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
ASIMD store, 4 element, one lane, size 32	VST4	5	2/3	V, L01	-
ASIMD store, 4 element, one lane, size 8/16	VST4	5	2/3	V, L01	-
(ASIMD store, writeback form)	-	(1)	-	+I	1

Notes:

1. Writeback forms of store instructions require an extra μ OP to update the base address. This update is typically performed in parallel with the store μ OP (update latency shown in parentheses).

3.24 Cryptography extensions

Table 3-39 AArch64 Cryptography extensions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Crypto AES ops	AESD, AESE, AESIMC, AESMC	2	2	V	-
Crypto polynomial (64x64) multiply long	PMULL (2)	2	1	V0	-
Crypto SHA1 hash acceleration op	SHA1H	2	1	V0	-
Crypto SHA1 hash acceleration ops	SHA1C, SHA1M, SHA1P	4	1	V0	-
Crypto SHA1 schedule acceleration ops	SHA1SU0, SHA1SU1	2	1	V0	-
Crypto SHA256 hash acceleration ops	SHA256H, SHA256H2	4	1	V0	-
Crypto SHA256 schedule acceleration ops	SHA256SU0, SHA256SU1	2	1	V0	-
Crypto SHA512 hash acceleration ops	SHA512H, SHA512H2, SHA512SU0, SHA512SU1	2	1	V0	-
Crypto SHA3 ops	BCAX, EOR3, RAX1, XAR	2	1	V0	-
Crypto SM3 ops	SM3PARTW1, SM3PARTW2SM3SS1, SM3TT1A, SM3TT1B, SM3TT2A, SM3TT2B	2	1	V0	-

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Crypto SM4 ops	SM4E, SM4EKEY	4	1	V0	-

Table 3-40 AArch32 Cryptography extensions

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Crypto AES ops	AESD, AESE, AESIMC, AESMC	2	2	V	1
Crypto polynomial (64x64) multiply long	VMULL.P64	2	1	V0	-
Crypto SHA1 hash acceleration op	SHA1H	2	1	V0	-
Crypto SHA1 hash acceleration ops	SHA1C, SHA1M, SHA1P	4	1	V0	-
Crypto SHA1 schedule acceleration ops	SHA1SU0, SHA1SU1	2	1	V0	-
Crypto SHA256 hash acceleration ops	SHA256H, SHA256H2	4	1	V0	-
Crypto SHA256 schedule acceleration ops	SHA256SU0, SHA256SU1	2	1	V0	-

Notes:

1. Adjacent AESE/AESMC instruction pairs and adjacent AESD/AESIMC instruction pairs will exhibit the performance characteristics described in Section 4.6.

3.25 CRC

Table 3-41 AArch64 CRC

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
CRC checksum ops	CRC32, CRC32C	2	1	M0	1

Table 3-42 AArch32 CRC

Instruction Group	AArch32 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
CRC checksum ops	CRC32, CRC32C	2	1	M0	1

Notes:

1. CRC execution supports late forwarding of the result from a producer μ OP to a consumer μ OP. This results in a 1 cycle reduction in latency as seen by the consumer.

3.26 SVE Predicate instructions

Table 3-43 SVE Predicate Instructions

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Loop control, based on predicate	BRKA, BRKB	2	2	M	1
Loop control, based on predicate and flag setting	BRKAS, BRKBS	3	2	M	1
Loop control, propagating	BRKN, BRKPA, BRKPB	2	1	M0	1
Loop control, propagating and flag setting	BRKNS, BRKPAS, BRKPBS	3	1	M0, M	1
Loop control, based on GPR	WHILEGE, WHILEGT, WHILEHI, WHILEHS, WHILELE, WHILELO, WHILELS, WHILELT, WHILERW, WHILEWR	3	1	M	-
Loop terminate	CTERMEQ, CTERMNE	1	1	M	-

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Predicate counting scalar	ADDPL, ADDVL, CNTB, CNTH, CNTW, CNTD, DECB, DECH, DECW, DECD, INCB, INCH, INCW, INCD, RDVL, SQDECB, SQDECH, SQDECW, SQDECD, SQINCB, SQINCH, SQINCW, SQINCD, UQDECB, UQDECH, UQDECW, UQDECD, UQINCB, UQINCH, UQINCW, UQINCD	2	2	M	-
Predicate counting scalar, ALL, {1,2,4}	INC, DEC	1	4	I	
Predicate counting scalar, active predicate	CNTP, DECP, INCP, SQDECP, SQINCP, UQDECP, UQINCP	2	2	M	-
Predicate counting vector, active predicate	DECP, INCP, SQDECP, SQINCP, UQDECP, UQINCP	7	1	M, M0, V	-
Predicate logical	AND, BIC, EOR, MOV, NAND, NOR, NOT, ORN, ORR	1	1	M0	1
Predicate logical, flag setting	ANDS, BICS, EORS, MOV, NANDS, NORS, NOTS, ORNS, ORRS	2	1	M0, M	1
Predicate reverse	REV	2	2	M	-
Predicate select	SEL	1	1	M0	-
Predicate set	PFALSE, PTRUE	2	2	M	-
Predicate set/initialize, set flags	PTRUES	3	2	M	-

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Predicate find first/next	PFIRST, PNEXT	3	2	M	-
Predicate test	PTEST	1	2	M	-
Predicate transpose	TRN1, TRN2	2	2	M	-
Predicate unpack and widen	PUNPKHI, PUNPKLO	2	2	M	-
Predicate zip/unzip	ZIP1, ZIP2, UZP1, UZP2	2	2	M	-

Notes:

1. When the governing predicate is the same as destination, the latency is increased by one cycle.

3.27 SVE integer instructions

Table 3-44 SVE integer instructions

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Arithmetic, absolute diff	SABD, UABD	2	2	V	-
Arithmetic, absolute diff accum	SABA, UABA	4(1)	1	V1	2
Arithmetic, absolute diff accum long	SABALB, SABALT, UABALB, UABALT	4(1)	1	V1	2
Arithmetic, absolute diff long	SABDLB, SABDLT, UABDLB, UABDLT	2	2	V	-

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Arithmetic, basic	ABS, ADD, ADR, CNOT, NEG, SADDLB, SADDLBT, SADDLT, SADDWB, SADDWT, SHADD, SHSUB, SHSUBR, SSUBLB, SSUBLBT, SSUBLT, SSUBLTB, SSUBWB, SSUBWT, SUB, SUBHNB, SUBHNT, SUBR, UADDLB, UADDLT, UADDWB, UADDWT, UHADD, UHSUB, UHSUBR, USUBLB, USUBLT, USUBWB, USUBWT	2	2	V	-
Arithmetic, complex	ADDHNB, ADDHNT, RADDHNB, RADDHNT, RSUBHNB, RSUBHNT, SQABS, SQADD, SQNEG, QSUB, QSUBR, SRHADD, SUQADD, UQADD, UQSUB, UQSUBR, USQADD, URHADD	2	2	V	-
Arithmetic, large integer	ADCLB, ADCLT, SBCLB, SBCLT	2	2	V	-
Arithmetic, pairwise add	ADDP	2	2	V	-
Arithmetic, pairwise add and accum long	SADALP, UADALP	4(1)	1	V1	2
Arithmetic, shift	ASR, ASRR, LSL, LSLR, LSR, LSRR	2	1	V1	-

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Arithmetic, shift and accumulate	SRSRA, SSRA, URSRA, USRA	4(1)	1	V1	2
Arithmetic, shift by immediate	SHRNB, SHRNT, SSHLLB, SSHLLT, USHLLB, USHLLT	2	1	V1	-
Arithmetic, shift by immediate and insert	SLI, SRI	2	1	V1	-
Arithmetic, shift complex	RSHRNB, RSHRNT, SQRSHL, SQRSHLR, SQRSHRNB, SQRSHRNT, SQRSHRUNB, SQRSHRUNT, SQSHL, SQSHLR, SQSHLU, SQSHRNB, SQSHRNT, SQSHRUNB, SQSHRUNT, UQRSHL, UQRSHLR, UQRSHRNB, UQRSHRNT, UQSHL, UQSHLR, UQSHRNB, UQSHRNT	4	1	V1	-
Arithmetic, shift right for divide	ASRD	4	1	V1	-
Arithmetic, shift rounding	SRSHL, SRSHLR, SRSHR, URSHL, URSHLR, URSHR	4	1	V1	-
Bit manipulation	BDEP, BEXT, BGRP	6	1/2	V1	-
Bitwise select	BSL, BSL1N, BSL2N, NBSL	2	2	V	-
Count/reverse bits	CLS, CLZ, CNT, RBIT	2	2	V	-
Broadcast logical bitmask immediate to vector	DUPM, MOV	2	2	V	-
Compare and set flags	CMPEQ, CMPGE, CMPGT, CMPHI, CMPHS, CMPL, CMPLO, CMPLS, CMPLT, CMPNE	4	1	V0, M	1

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Complex add	CADD, SQCADD	2	2	V	-
Complex dot product 8-bit element	CDOT	3(1)	2	V	2
Complex dot product 16-bit element	CDOT	4(1)	1	V0	2
Complex multiply-add B, H, S element size	CMLA	4(1)	1	V0	2
Complex multiply-add D element size	CMLA	5(3)	1/2	V0	2
Conditional extract operations, scalar form	CLASTA, CLASTB	8	1	M0, V1, V	-
Conditional extract operations, SIMD&FP scalar and vector forms	CLASTA, CLASTB, COMPACT, SPLICE	3	1	V1	-
Convert to floating point, 64b to float or convert to double	SCVTF, UCVTF	3	1	V0	-
Convert to floating point, 32b to single or half	SCVTF, UCVTF	4	1/2	V0	-
Convert to floating point, 16b to half	SCVTF, UCVTF	6	1/4	V0	-
Copy, scalar	CPY	5	1	M0, V	
Copy, scalar SIMD&FP or imm	CPY	2	2	V	
Divides, 32 bit	SDIV, SDIVR, UDIV, UDIVR	7 to 12	1/11 to 1/7	V0	3
Divides, 64 bit	SDIV, SDIVR, UDIV, UDIVR	7 to 20	1/20 to 1/7	V0	3
Dot product, 8 bit	SDOT, UDOT	3(1)	2	V	2
Dot product, 8 bit, using signed and unsigned integers	SUDOT, USDOT	3(1)	2	V	2
Dot product, 16 bit	SDOT, UDOT	4(1)	1	V0	2
Duplicate, immediate and indexed form	DUP, MOV	2	2	V	-
Duplicate, scalar form	DUP, MOV	3	1	M0	-
Extend, sign or zero	SXTB, SXTW, UXTB, UXTH, UXTW	2	1	V1	-
Extract	EXT	2	2	V	-

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Extract narrow saturating	SQXTNB, SQXTNT, SQXTUNB, SQXTUNT, UQXTNB, UQXTNT	4	1	V1	-
Extract/insert operation, SIMD and FP scalar form	LASTA, LASTB, INSR	3	1	V1	-
Extract/insert operation, scalar	LASTA, LASTB, INSR	5	1	V1, M0	-
Histogram operations	HISTCNT, HISTSEG	2	2	V	-
Horizontal operations, B, H, S form, immediate operands only	INDEX	4	1	V0	-
Horizontal operations, B, H, S form, scalar, immediate operands)/ scalar operands only / immediate, scalar operands	INDEX	7	1	M0, V0	-
Horizontal operations, D form, immediate operands only	INDEX	5	1/2	V0	-
Horizontal operations, D form, scalar, immediate operands)/ scalar operands only / immediate, scalar operands	INDEX	8	1/2	M0, V0	-
Logical	AND, BIC, EON, EOR, EORBT, EORTB, MOV, NOT, ORN, ORR	2	2	V	-
Max/min, basic and pairwise	SMAX, SMAXP, SMIN, SMINP, UMAX, UMAXP, UMIN, UMINP	2	2	V	-
Matching operations	MATCH, NMATCH	2	1	V0, M	1,5
Matrix multiply-accumulate	SMMLA, UMMLA, USMMLA	3(1)	2	V	2
Move prefix	MOVPRFX	2	2	V	-
Multiply, B, H, S element size	MUL, SMULH, UMULH	4	1	V0	-
Multiply, D element size	MUL, SMULH, UMULH	5	1/2	V0	-

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Multiply long	SMULLB, SMULLT, UMULLB, UMULLT	4	1	V0	-
Multiply accumulate, B, H, S element size	MLA, MLS	4(1)	1	V0	2
Multiply accumulate, D element size	MLA, MLS, MAD, MSB,	5(3)	1/2	V0	2
Multiply accumulate long	SMLALB, SMLALT, SMLSLB, SMLSLT, UMLALB, UMLALT, UMLSLB, UMLSLT	4(1)	1	V0	2
Multiply accumulate saturating doubling long regular	SQDMLALB, SQDMLALT, SQDMLALBT, SQDMLSLB, SQDMLSLT, SQDMLSLBT	4(2)	1	V0	4
Multiply saturating doubling high, B, H, S element size	SQDMULH	4	1	V0	-
Multiply saturating doubling high, D element size	SQDMULH	5	1/2	V0	-
Multiply saturating doubling long	SQDMULLB, SQDMULLT	4	1	V0	-
Multiply saturating rounding doubling regular/complex accumulate, B, H, S element size	SQRDMLAH, SQRDMLSH, SQRDCMLAH	4(2)	1	V0	4
Multiply saturating rounding doubling regular/complex accumulate, D element size	SQRDMLAH, SQRDMLSH, SQRDCMLAH	5(3)	1/2	V0	4
Multiply saturating rounding doubling regular/complex, B, H, S element size	SQRDMULH	4	1	V0	-
Multiply saturating rounding doubling regular/complex, D element size	SQRDMULH	5	1/2	V0	-
Multiply/multiply long, (8x8) polynomial	PMUL, PMULLB, PMULLT	2	1	V0	-

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Predicate counting vector	CNT, DECB, DECH, DECW, DECD, INCB, INCH, INCW, INCD, SQDECB, SQDECH, SQDECW, SQDECD, SQINCB, SQINCH, SQINCW, SQINCD, UQDECB, UQDECH, UQDECW, UQDECD, UQINCB, UQINCH, UQINCW, UQINCD	2	2	V0	-
Reciprocal estimate	URECPE, URSQRTE	4	1/2	V0	
Reduction, arithmetic, B form	SADDV, UADDV, SMAXV, SMINV, UMAXV, UMINV	11	1/2	V, V1	-
Reduction, arithmetic, H form	SADDV, UADDV, SMAXV, SMINV, UMAXV, UMINV	9	1/2	V, V1	-
Reduction, arithmetic, S form	SADDV, UADDV, SMAXV, SMINV, UMAXV, UMINV	8	4/5	V, V1	-
Reduction, logical	ANDV, EORV, ORV	6	1	V, V1	-
Reverse, vector	REV, REVB, REVH, REVW	2	2	V	-
Select, vector form	MOV, SEL	2	2	V	-
Table lookup	TBL	2	2	V	-
Table lookup extension	TBX	2	2	V	-
Transpose, vector form	TRN1, TRN2	2	2	V	-
Unpack and extend	SUNPKHI, SUNPKLO, UUNPKHI, UUNPKLO	2	2	V	-
Zip/unzip	UZP1, UZP2, ZIP1, ZIP2	2	2	V	-

Notes:

1. When the governing predicate is the same as destination, the latency is increased by one cycle.
2. SVE accumulate pipelines support late-forwarding of accumulate operands from similar μ OPs, allowing a typical sequence of such μ OPs to issue one every N cycles (accumulate latency N shown in parentheses).
3. SVE integer divide operations are performed using an iterative algorithm and block subsequent similar operations to the same pipeline until complete.
4. Same as 2 except that for saturating instructions require an extra cycle of latency for late-forwarding accumulate operands.
5. If the consuming instruction has a flag source, the latency for this instruction is 4 cycles.

3.28 SVE floating-point instructions

Table 3-45 SVE floating-point instructions

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Floating point absolute value/difference	FABD, FABS	2	2	V	-
Floating point arithmetic	FADD, FADDP, FNEG, FSUB, FSUBR	2	2	V	-
Floating point associative add, F16	FADDA	10	1/9	V1	-
Floating point associative add, F32	FADDA	6	1/5	V1	-
Floating point associative add, F64	FADDA	4	2	V	-
Floating point compare	FACGE, FACGT, FACLE, FACLT, FCMEQ, FCMGE, FCMGT, FCMLE, FCMLT, FCMNE, FCMUO	2	1	V0	-
Floating point complex add	FCADD	3	2	V	-
Floating point complex multiply add	FCMLA	5(2)	2	V	1
Floating point convert, long or narrow (F16 to F32 or F32 to F16)	FCVT, FCVTLT, FCVTNT	4	1/2	V0	-
Floating point convert, long or narrow (F16 to F64, F32 to F64, F64 to F32 or F64 to F16)	FCVT, FCVTLT, FCVTNT	3	1	V0	-
Floating point convert, round to odd	FCVTX, FCVTXNT	3	1	V0	-

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Floating point base2 log, F16	FLOGB	6	1/4	V0	
Floating point base2 log, F32	FLOGB	4	1/2	V0	
Floating point base2 log, F64	FLOGB	3	1	V0	
Floating point convert to integer, F16	FCVTZS, FCVTZU	6	1/4	V0	-
Floating point convert to integer, F32	FCVTZS, FCVTZU	4	1/2	V0	-
Floating point convert to integer, F64	FCVTZS, FCVTZU	3	1	V0	-
Floating point copy	FCPY, FDUP, FMOV	2	2	V	-
Floating point divide, F16	FDIV, FDIVR	10 to 13	1/12 to 1/10	V0	2
Floating point divide, F32	FDIV, FDIVR	7 to 10	1/9 to 1/7	V0	2
Floating point divide, F64	FDIV, FDIVR	7 to 15	1/14 to 1/7	V0	2
Floating point min/max pairwise	FMAXP, FMAXNMP, FMINP, FMINNMP	2	2	V	
Floating point min/max	FMAX, DMIN, FMAXNM, FMINNMM	2	2	V	-
Floating point multiply	FSCALE, FMUL, FMULX	3	2	V	-
Floating point multiply accumulate	FMLA, FMLS, FMAD, FMSB, FNMAD, FNMLA, FNMLS, FNMSB	4(2)	2	V	1
Floating point multiply add/sub accumulate long	FMLALB, FMLALT, FMLSBLB, FMLSBLT	4(2)	2	V	1
Floating point reciprocal estimate, F16	FRECPE, FRECPX, FRSQRTE	6	1/4	V0	-
Floating point reciprocal estimate, F32	FRECPE, FRECPX, FRSQRTE	4	1/2	V0	-
Floating point reciprocal estimate, F64	FRECPE, FRECPX, FRSQRTE	3	1	V0	-
Floating point reciprocal step	FRECPS, FRSQRSTS	4	2	V	-
Floating point reduction, F16	FADDV, FMAXNMV, FMAXV, FMINNMMV, FMINV	6	2/3	V	-

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Floating point reduction, F32	FADDV, FMAXNMV, FMAXV, FMINNMV, FMINV	4	1	V	-
Floating point reduction, F64	FADDV, FMAXNMV, FMAXV, FMINNMV, FMINV	2	2	V	-
Floating point round to integral, F16	FRINTA, FRINTM, FRINTN, FRINTP, FRINTX, FRINTZ	6	1/4	V0	-
Floating point round to integral, F32	FRINTA, FRINTM, FRINTN, FRINTP, FRINTX, FRINTZ	4	1/2	V0	-
Floating point round to integral, F64	FRINTA, FRINTM, FRINTN, FRINTP, FRINTX, FRINTZ	3	1	V0	-
Floating point square root, F16	FSQRT	10 to 13	1/12 to 1/10	V0	2
Floating point square root, F32	FSQRT	7 to 10	1/9 to 1/7	V0	2
Floating point square root F64	FSQRT	7 to 16	1/14 to 1/7	V0	2
Floating point trigonometric exponentiation	FEXPA	3	1	V1	
Floating point trigonometric multiply add	FTMAD	4	2	V	
Floating point trigonometric, miscellaneous	FTSMUL, FTSEL	3	2	V	-

Notes:

1. SVE multiply-accumulate pipelines support late-forwarding of accumulate operands from similar μ OPs, allowing a typical sequence of floating-point multiply-accumulate μ OPs to issue one every N cycles (accumulate latency N shown in parentheses).
2. SVE divide and square root operations are performed using an iterative algorithm and block subsequent similar operations to the same pipeline until complete.

3.29 SVE BFloat16 (BF16) instructions

Table 3-46 SVE Bfloat16 (BF16) instructions

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Convert, F32 to BF16	BFCVT, BFCVTNT	3	1	V0	-

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Dot product	BFDOT	4(2)	2	V	1
Matrix multiply accumulate	BFMMLA	5(3)	2	V	1
Multiply accumulate long	BFMLALB, BFMLALT	4(2)	2	V	1

Notes:

1. SVE pipelines that execute these instructions support late-forwarding of accumulate operands from similar μ OPs, allowing a typical sequence of μ OPs to issue one every N cycles (accumulate latency N shown in parentheses).

3.30 SVE Load instructions

The latencies shown assume the memory access hits in the Level 1 Data Cache and represent the maximum latency to load all the vector registers written by the instruction.

Table 3-47 SVE Load instructions

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Load vector	LDR	6	3	L	-
Load predicate	LDR	6	3	L, M	-
Contiguous load, scalar + imm	LD1B, LD1D, LD1H, LD1W, LD1SB, LD1SH, LD1SW,	6	3	L	-
Contiguous load, scalar + scalar	LD1B, LD1D, LD1H, LD1W, LD1SB, LD1SH LD1SW	6	3	L01	-
Contiguous load broadcast, scalar + imm	LD1RB, LD1RH, LD1RD, LD1RW, LD1RSB, LD1RSH, LD1RSW, LD1RQB, LD1RQD, LD1RQH,	6	3	L	-
Contiguous load broadcast, scalar + scalar	LD1RQB, LD1RQD, LD1RQH, LD1RQW	6	3	L	-
Non temporal load, scalar + imm	LDNT1B, LDNT1D, LDNT1H, LDNT1W	6	3	L	-

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Non temporal load, scalar + scalar	LDNT1B, LDNT1D, LDNT1H LDNT1W	6	3	L, S	-
Non temporal gather load, vector + scalar 32-bit element size	LDNT1B, LDNT1H, LDNT1W, LDNT1SB, LDNT1SH	9	1	L, V	-
Non temporal gather load, vector + scalar 64-bit element size	LDNT1B, LDNT1D, LDNT1H, LDNT1W, LDNT1SB, LDNT1SH, LDNT1SW	10	1/2	L, V1	-
Contiguous first faulting load, scalar + scalar	LDF1B, LDF1D, LDF1H, LDF1W, LDF1SB, LDF1SD, LDF1SH LDF1SW	6	3	L, S	-
Contiguous non faulting load, scalar + imm	LDNF1B, LDNF1D, LDNF1H, LDNF1W, LDNF1SB, LDNF1SH, LDNF1SW	6	3	L	-
Contiguous Load two structures to two vectors, scalar + imm	LD2B, LD2D, LD2H, LD2W	8	1	V, L	-
Contiguous Load two structures to two vectors, scalar + scalar	LD2B, LD2D, LD2H, LD2W	9	1	V, L	-
Contiguous Load three structures to three vectors, scalar + imm	LD3B, LD3D, LD3H, LD3W	9	3/2	V, L	-
Contiguous Load three structures to three vectors, scalar + scalar	LD3B, LD3D, LD3H, LD3W	10	3/2	V, L, S	-
Contiguous Load four structures to four vectors, scalar + imm	LD4B, LD4D, LD4H LD4W	9	1/2	V, L	-

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Contiguous Load four structures to four vectors, scalar + scalar	LD4B, LD4D, LD4H, LD4W	10	1/2	L, V, S	-
Gather load, vector + imm, 32-bit element size	LD1B, LD1H, LD1W, LD1SB, LD1SH, LD1SW, LDFF1B, LDFF1H, LDFF1W, LDFF1SB, LDFF1SH, LDFF1SW	9	1	L, V	-
Gather load, vector + imm, 64-bit element size	LD1B, LD1D, LD1H, LD1W, LD1SB, LD1SH, LD1SW, LDFF1B, LDFF1D, LDFF1H, LDFF1W, LDFF1SB, LDFF1SD, LDFF1SH, LDFF1SW	9	1/2	L, V	-
Gather load, 32-bit scaled offset	LD1H, LD1SH, LDFF1H, LDFF1SH, LD1W, LDFF1W, LDFF1SW	10	1/2	L, V	-
Gather load, 32-bit unpacked unscaled offset	LD1B, LD1SB, LDFF1B, LDFF1SB, LD1D, LDFF1D, LD1H, LD1SH, LDFF1H, LDFF1SH, LD1W, LD1SW, LDFF1W, LDFF1SW	9	1	L, V	-

3.31 SVE Store instructions

Table 3-48 SVE Store instructions

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Store from predicate reg	STR	1	2	L01	-
Store from vector reg	STR	2	2	L01, V	-

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Contiguous store, scalar + imm	ST1B, ST1H, ST1D, ST1W	2	2	L01, V	-
Contiguous store, scalar + scalar	ST1H	2	2	L01, S, V	-
Contiguous store, scalar + scalar	ST1B, ST1D, ST1W	2	2	L01, V	-
Contiguous store two structures from two vectors, scalar + imm	ST2B, ST2H, ST2D, ST2W	4	1	L01, V	-
Contiguous store two structures from two vectors, scalar + scalar	ST2H	4	1	L01, S, V	-
Contiguous store two structures from two vectors, scalar + scalar	ST2B, ST2D, ST2W	4	1	L01, V	-
Contiguous store three structures from three vectors, scalar + imm	ST3B, ST3D, ST3H, ST3W	7	2/9	L01, V	-
Contiguous store three structures from three vectors, scalar + scalar	ST3H	7	2/9	L01, S, V	-
Contiguous store three structures from three vectors, scalar + scalar	ST3B, ST3D, ST3W	7	2/9	L01, S, V	-
Contiguous store four structures from four vectors, scalar + imm	ST4B, ST4D, ST4H, ST4W	11	1/9	L01, V	-
Contiguous store four structures from four vectors, scalar + scalar	ST4H	11	1/9	L01, S, V	-
Contiguous store four structures from four vectors, scalar + scalar	ST4B, ST4D, ST4W	11	1/9	L01, S, V	-
Non temporal store, scalar + imm	STNT1B, STNT1D, STNT1H, STNT1W	2	2	L01, V	-
Non temporal store, scalar + scalar	STNT1H	2	2	L01, S, V	-
Non temporal store, scalar + scalar	STNT1B, STNT1D, STNT1W	2	2	L01, V	-
Scatter non temporal store, vector + scalar 32-bit element size	STNT1B, STNT1H, STNT1W	4	1/2	L01, V	-

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Scatter non temporal store, vector + scalar 64-bit element size	STNT1B, STNT1D, STNT1H, STNT1W	2	1	L01, V	-
Scatter store vector + imm 32-bit element size	ST1B, ST1H, ST1W	4	1/2	L01, V	-
Scatter store vector + imm 64-bit element size	ST1B, ST1D, ST1H, ST1W	2	1	L01, V	-
Scatter store, 32-bit scaled offset	ST1H, ST1W	4	1/2	L01, V	-
Scatter store, 32-bit unpacked unscaled offset	ST1B, ST1D, ST1H, ST1W	2	1	L01, V	-
Scatter store, 32-bit unpacked scaled offset	ST1D, ST1H, ST1W	2	1	L01, V	-
Scatter store, 32-bit unscaled offset	ST1B, ST1H, ST1W	4	1/2	L01, V	-
Scatter store, 64-bit scaled offset	ST1D, ST1H, ST1W	2	1	L01, V	-
Scatter store, 64-bit unscaled offset	ST1B, ST1D, ST1H, ST1W	2	1	L01, V	-

3.32 SVE Miscellaneous instructions

Table 3-49 SVE miscellaneous instructions

Instruction Group	SVE Instruction	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Read first fault register, unpredicated	RDFFR	2	1	M0	-
Read first fault register, predicated	RDFFR	3	1	M0, M	1
Read first fault register and set flags	RDFFRS	4	1/2	M0, M	1
Set first fault register	SETFFR	2	1	M0	-
Write to first fault register	WRFFR	2	1	M0	-

Notes:

1. When destination is same as the governing predicate, the latency of the instruction increases by one cycle.

3.33 SVE Cryptographic instructions

Table 3-50 SVE cryptographic instructions

Instruction Group	AArch64 Instructions	Exec Latency	Execution Throughput	Utilized Pipelines	Notes
Crypto AES ops	AESD, AESE, AESIMC, AESMC	2	2	V	-
Crypto SHA3 ops	BCAX, EOR3, RAX1, XAR	2	1	V0	-
Crypto SM4 ops	SM4E, SM4EKEY	4	1	V0	-

4 Special considerations

4.1 Dispatch constraints

Dispatch of μ OPs from the in-order portion to the out-of-order portion of the microarchitecture includes several constraints. It is important to consider these constraints during code generation to maximize the effective dispatch bandwidth and subsequent execution bandwidth of Cortex-A710.

The dispatch stage can process up to 5 MOPs per cycle and dispatch up to 10 μ OPs per cycle, with the following limitations on the number of μ OPs of each type that may be simultaneously dispatched.

- Up to 4 μ OPs utilizing the S or B pipelines
- Up to 4 μ OPs utilizing the M pipelines
- Up to 2 μ OPs utilizing the M0 pipelines
- Up to 2 μ OPs utilizing the V0 pipeline
- Up to 2 μ OPs utilizing the V1 pipeline
- Up to 6 μ OPs utilizing the L pipelines

In the event there are more μ OPs available to be dispatched in a given cycle than can be supported by the constraints above, μ OPs will be dispatched in oldest to youngest age-order to the extent allowed by the above.

4.2 Dispatch stall

In the event of a V-pipeline μ OP containing more than 1 quad-word register source, a portion or all of which was previously written as one or multiple single words, that μ OP will stall in dispatch for three cycles. This stall occurs only on the first such instance, and subsequent consumers of the same register will not experience this stall.

4.3 Optimizing general-purpose register spills and fills

Register transfers between general-purpose registers (GPR) and ASIMD registers (VPR) are lower latency than reads and writes to the cache hierarchy, thus it is recommended that GPR registers be filled/spilled to the VPR rather to memory, when possible.

4.4 Optimizing memory routines

To achieve maximum throughput for memory copy (or similar loops), one should do the following.

Unroll the loop to include multiple load and store operations per iteration, minimizing the overheads of looping.

Align stores on 32B boundary wherever possible.

Use non-writeback forms of LDP and STP instructions interleaving them like shown in the example below:

```

Loop_start:
    SUBS    x2, x2, #96
    LDP     q3, q4, [x1, #0]
    STP     q3, q4, [x0, #0]
    LDP     q3, q4, [x1, #32]
    STP     q3, q4, [x0, #32]
    LDP     q3, q4, [x1, #64]
    STP     q3, q4, [x0, #64]
    ADD     x1, x1, #96
    ADD     x0, x0, #96
    BGT     Loop_start
  
```

A recommended copy routine for AArch32 would look like the sequence above but would use LDRD/STRD instructions. Avoid load-/store-multiple instruction encodings (such as LDM and STM).

If the memory locations being copied are non-cacheable, the non-temporal version of LDPQ (LDNPQ) should be used. STPQ should still be used for the stores.

Similarly, it is recommended to use LDPQ to achieve maximum throughput for memcmp (memory compare) loops that compare cacheable memory. LDNPQ should be used for non-cacheable memory.

To achieve maximum throughput on memset, it is recommended that one do the following.

Unroll the loop to include multiple store operations per iteration, minimizing the overheads of looping.

```

Loop_start:
    STP     q1, q3, [x0, #0]
    STP     q1, q3, [x0, #0x20]
    STP     q1, q3, [x0, #0x40]
    STP     q1, q3, [x0, #0x60]
    ADD     x0, x0, #0x80
    SUBS    x2, x2, #0x80
    B.GT    Loop_start
  
```

To achieve maximum performance on memset to zero, it is recommended that one use DC ZVA instead of STP. An optimal routine might look something like the following.

```

Loop_start:
  
```

```

SUBS    x2, x2, #0x80
DC      ZVA, x0
ADD     x0, x0, #0x40
DC      ZVA, x0
ADD     x0, x0, #0x40
B.GT    Loop_start

```

4.5 Load/Store alignment

The Armv8-A architecture allows many types of load and store accesses to be arbitrarily aligned. The Cortex-A710 core handles most unaligned accesses without performance penalties. However, there are cases which could reduce bandwidth or incur additional latency, as described below.

- Load operations that cross a cache-line (64-byte) boundary.
- Quad-word load operations that are not 4B aligned.
- Store operations that cross a 32B boundary.

4.6 AES encryption/decryption

Cortex-A710 can issue two AESE/AESMC/AESD/AESIMC instruction every cycle (fully pipelined) with an execution latency of two cycles. This means encryption or decryption for at least four data chunks should be interleaved for maximum performance:

```

AESE    data0, key_reg
AESMC   data0, data0
AESE    data1, key_reg
AESMC   data1, data1
AESE    data2, key_reg
AESMC   data2, data2
AESE    data3, key_reg
AESMC   data3, data3
AESE    data0, key_reg
AESMC   data0, data0
...

```

Pairs of dependent AESE/AESMC and AESD/AESIMC instructions are higher performance when they are adjacent in the program code and both instructions use the same destination register.

4.7 Region based fast forwarding

The forwarding logic in the V pipelines is optimized to provide optimal latency for instructions which are expected to commonly forward to one another. The effective latency of FP and ASIMD instructions as described in section 3 is increased by one cycle if the producer and consumer instructions are not part of the same forwarding region. These optimized forwarding regions are defined in the following table.

Table 4-1 Optimized forwarding regions

Region	Instruction Types	Notes
1	ASIMD/SVE integer ALU, ASIMD/SVE integer shift, ASIMD/scalar insert and move, ASIMD/SVE integer abs/cmp/max/min and the ASIMD miscellaneous instructions in table 3-18.	1
2	FP/ASIMD/SVE floating-point multiply, FP/ASIMD/SVE floating point multiply-accumulate, FP/ASIMD/SVE compare, FP/ASIMD/SVE add/sub and the ASIMD miscellaneous instructions in table 3-18.	1,2,3
3	ASIMD/SVE Crypto and SHA1/SHA256	-
4	ASIMD/SVE AES, ASIMD/SVE polynomial multiply and all the instruction types in region 1.	1
5	ASIMD/SVE BFDOT and BFMMMLA instructions	-

Notes:

1. Reciprocal step and estimate instructions are excluded from this region.
2. ASIMD/SVE extract narrow, saturating instructions are excluded from this region.
3. ASIMD miscellaneous instructions can only be consumers of this region.

The following instructions are not a part of any region:

- FP/ASIMD/SVE floating-point div/sqrt and SVE integer divides
- FP/ASIMD/SVE convert and rounding instructions that do not write to general purpose registers
- ASIMD/SVE integer mul/mac
- ASIMD/SVE integer reduction

In addition to the regions mentioned in the table above, all instructions in regions 1 and 2 can fast forward to FP/ASIMD/SVE stores, FP/ASIMD vector to integer register transfers and ASIMD converts that write to general purpose registers.

More special notes about the forwarding region in table 4-1:

- Element sources (the non-vector operand in "by element" multiplies) used by ASIMD/SVE floating-point multiply and multiply-accumulate operations cannot be consumers.

- Complex shift by immediate/register and shift accumulate instructions cannot be producers (see sections 3.16 and 3.25) in region 1.
- Extract narrow, saturating instructions cannot be producers (see sections 3.19 and 3.25) in region 1.
- Absolute difference accumulate and pairwise add and accumulate instructions cannot be producers (see sections 3.16 and 3.25) in region 1.
- For floating-point producer-consumer pairs, the precision of the instructions should match (single, double or half) in region 2.
- Pair-wise floating-point instructions cannot be producers or consumers in region 2.

It is not advisable to interleave instructions belonging to different regions. Also, certain instructions can only be producers or consumers in a particular region but not both (see footnote 3 for table 4-1). For example, the code below interleaves producers and consumers from regions 1 and 2. This will result in an additional latency of 1 cycle as seen by FMUL.

FSUB v27.2s, v28.2s, v20.2s – Region 2

FADD v20.2s, v28.2s, v20.2s – Region 2

MOV v27.s[1], v20.s[1] - Region 2 producer but not a region 2 consumer

FMUL v26.2s, v27.2s, v6.2s – Region 2

4.8 Branch instruction alignment

Branch instruction and branch target instruction alignment and density can affect performance.



For best case performance, avoid placing more than four branch instructions within an aligned 32-byte instruction memory region.

4.9 FPCR self-synchronization

Programmers and compiler writers should note that writes to the FPCR register are self-synchronizing, i.e. its effect on subsequent instructions can be relied upon without an intervening context synchronizing operation.

4.10 Special register access

The Cortex-A710 core performs register renaming for general purpose registers to enable speculative and out-of-order instruction execution. But most special-purpose registers are not renamed. Instructions that read or write non-renamed registers are subjected to one or more of the following additional execution constraints.

Non-Speculative Execution – Instructions may only execute non-speculatively.

In-Order Execution – Instructions must execute in-order with respect to other similar instructions or in some cases all instructions.

Flush Side-Effects – Instructions trigger a flush side-effect after executing for synchronization.

The table below summarizes various special-purpose register read accesses and the associated execution constraints or side-effects.

Table 4-2 Special-purpose register read accesses

Register Read	Non-Speculative	In-Order	Flush Side-Effect	Notes
APSR	Yes	Yes	No	3
CurrentEL	No	Yes	No	-
DAIF	No	Yes	No	-
DLR_EL0	No	Yes	No	-
DSPSR_EL0	No	Yes	No	-
ELR_*	No	Yes	No	-
FPCR	No	Yes	No	-
FPSCR	Yes	Yes	No	2
FPSR	Yes	Yes	No	2
NZCV	No	No	No	1
SP_*	No	No	No	1
SPSel	No	Yes	No	-
SPSR_*	No	Yes	No	-
FFR	No	Yes	No	-

Notes:

1. The NZCV and SP registers are fully renamed.
2. FPSR/FPSCR reads must wait for all prior instructions that may update the status flags to execute and retire.
3. APSR reads must wait for all prior instructions that may set the Q bit to execute and retire.
4. The table below summarizes various special-purpose register write accesses and the associated execution constraints or side-effects.

Table 4-3 Special-purpose register write accesses

Register Write	Non-Speculative	In-Order	Flush Side-Effect	Notes
APSR	Yes	Yes	No	4
DAIF	Yes	Yes	No	-
DLR_EL0	Yes	Yes	No	-
DSPSR_EL0	Yes	Yes	No	-

Register Write	Non-Speculative	In-Order	Flush Side-Effect	Notes
ELR_*	Yes	Yes	No	-
FPCR	Yes	Yes	Maybe	2
FPSCR	Yes	Yes	Maybe	2, 3
FPSR	Yes	Yes	No	3
NZCV	No	No	No	1
SP_*	No	No	No	1
SPSel	Yes	Yes	Yes	-
SPSR_*	Yes	Yes	No	-
FFR	Yes	Yes	No	-

Notes:

1. The NZCV and SP registers are fully renamed.
2. If the FPCR/FPSCR write is predicted to change the control field values, it will introduce a barrier which prevents subsequent instructions from executing. If the FPCR/FPSCR write is predicted to not change the control field values, it will execute without a barrier but trigger a flush if the values change.
3. FPSR/FPSCR writes must stall at dispatch if another FPSR/FPSCR write is still pending.
4. APSR writes that set the Q bit will introduce a barrier which prevents subsequent instructions from executing until the write completes.

4.11 Register forwarding hazards

The Armv8-A architecture allows FP/ASIMD instructions to read and write 32-bit S-registers. In AArch32, each S-register corresponds to one half (upper or lower) of an overlaid 64-bit D-register. A Q register in turn consists of two overlaid D registers. Register forwarding hazards may occur when one μ OP reads a Q-register operand that has recently been written with one or more S-register results. Consider the following scenario.

```
VADD    S0, S1, S2
VADD    Q6, Q5, Q0
```

The first instruction writes S0, which corresponds to the lowest part of Q0. The second instruction then requires Q0 as an input operand. In this scenario, there is a RAW dependency between the first and the second instructions. In most cases, Cortex-A710 performs slightly worse in such situations.

Cortex-A710 is able to avoid this register-hazard condition for certain cases. The following rules describe the conditions under which a register-hazard can occur.

- The producer writes an S-register (not a D[x] scalar)
- The consumer reads an overlapping Q-register (not as a D[x] scalar)
- The consumer is a FP/ASIMD μ OP (not a store or MOV μ OP)

To avoid unnecessary hazards, it is recommended that the programmer use D[x] scalar writes when populating registers prior to ASIMD operations. For example, either of the following instruction forms would safely prevent a subsequent hazard.

```
VLD1.32    D0[x], [address]
VADD       Q1, Q0, Q2
```

4.12 IT blocks

The Armv8-A architecture performance deprecates some uses of the IT instruction in such a way that software may be written using multiple naïve single instruction IT blocks. It is preferred that software instead generate multi instruction IT blocks rather than single instruction blocks.

4.13 Instruction fusion

Cortex-A710 can accelerate certain instruction pairs in an operation called fusion. Specific Aarch64 instruction pairs that can be fused are as follows:

CMP/CMN (immediate) + B.cond

CMP/CMN (register) + B.cond

CMP (immediate) + CSEL

CMP (register) + CSEL

CMP (immediate) + CSET

CMP (register) + CSET

TST (immediate) + B.cond

TST (register) + B.cond

BICS (register) + B.cond

NOP + Any instruction

The following instruction pairs are fused in both Aarch32 and Aarch64 modes:

AESE + AESMC (see Section 4.6 on AES Encryption/Decryption)

AESD + AESIMC (see Section 4.6 on AES Encryption/Decryption)

CMP/CMN (immediate) + B.cond

CMP/CMN (register) + B.cond

TST (immediate) + B.cond

TST (register) + B.cond

BICS (register) + B.cond

These instruction pairs must be adjacent to each other in program code. For CMP, CMN, TST and BICS, fusion is not allowed for shifted and/or extended register forms. For BICS, the destination register should be XZR or WZR if fusion is to take place.

4.14 Zero Latency MOVs

A subset of register-to-register move operations and move immediate operations are executed with zero latency. These instructions do not utilize the scheduling and execution resources of the machine. These are as follows:

MOV Xd, #0

MOV Xd, XZR

MOV Wd, #0

MOV Wd, WZR

MOV Hd, WZR

MOV Hd, XZR

MOV Sd, WZR

MOV Dd, XZR

MOVI Dd, #0

MOVI Vd.2D, #0

MOV Rd, #0 (AArch32)

MOV Wd, Wn

MOV Xd, Xn

MOV Rd, Rn (AArch32)

The last 3 instructions may not be executed with zero latency under certain conditions.

4.15 Cache maintenance operation

While using set way invalidation operations on L1 cache, it is recommended that software be written to traverse the sets in the inner loop and ways in the out loop.

4.16 Memory Tagging - Tagging Performance

To achieve maximum throughput for tag-only, it is recommended that one do the following.

Unroll the loop to include multiple store operations per iteration, minimizing the overheads of looping. Use STGM (or DCGVA) instruction as shown in the example below:

```
Loop_start:
SUBS    x2, x2, #0x80
STGM    x1, [x0]
ADD     x0, x0, #0x40
STGM    x1, [x0]
ADD     x0, x0, #0x40
B.GT    Loop_start
```

To achieve maximum throughput for tag and zeroing out data, it is recommended that one do the following.

Unroll the loop to include multiple store operations per iteration, minimizing the overheads of looping. Use STZGM (or DCZGVA) instruction as shown in the example below:

```
Loop_start:
SUBS    x2, x2, #0x80
STZGM   x1, [x0]
ADD     x0, x0, #0x40
STZGM   x1, [x0]
ADD     x0, x0, #0x40
B.GT    Loop_start
```

To achieve maximum throughput for tag-loading, it is recommended that one do the following.

Unroll the loop to include multiple load operations per iteration, minimizing the overheads of looping. Use LDGM instruction as shown in the example below:

```
Loop_start:
SUBS    x2, x2, #0x80
LDGM    x1, [x0]
ADD     x0, x0, #0x40
LDGM    x1, [x0]
```

```
ADD    x0, x0, #0x40
B.GT   Loop_start
```

Also, it is recommended to use STZGM (or DCZGVA) to set tag if data is not a concern.

4.17 Memory Tagging - Synchronous Mode

In synchronous tag checking mode, stores cannot be performed speculatively. Each store must complete a tag check before the next store can be executed non-speculatively. Thus, performance of stores in synchronous tag checking mode will be diminished.

It is recommended to use asynchronous mode for better performance.

4.18 Complex ASIMD and SVE instructions

The bandwidth of the following ASIMD and SVE instructions is limited by decode constraints and it is advisable to avoid them when high performing code is desired.

ASIMD

LD4R, post-indexed addressing, element size = 64b.

LD4, single 4-element structure, post indexed addressing mode, element size = 64b.

LD4, multiple 4-element structures, quad form.

LD4, multiple 4-element structures, double word form.

ST4, multiple 4-element structures, quad form, element size less than 64b.

ST4, multiple 4-element structures, quad form, element size = 64b, post indexed addressing mode.

SVE

LD1B gather (scalar + vector addressing) where vector index register is the same as the destination register and element size = 32. Addressing mode is 32b unscaled offset.

LD1H gather (scalar + vector addressing) where vector index register is the same as the destination register and element size = 32. Addressing mode is 32b scaled or unscaled offset.

LD1W gather (scalar + vector addressing) where vector index register is the same as the destination register and element size = 32. Addressing mode is 32b scaled or unscaled offset.

LD3[B/H/W/D] contiguous (scalar + scalar addressing).

LD4[B/H/D/W] contiguous (scalar + immediate addressing).

LD4[B/H/D/W] contiguous (scalar + scalar addressing).

LDFF1B gather (scalar + vector addressing) where vector index register is the same as the destination register and element size = 32. Addressing mode is 32b unscaled offset.

LDFF1H gather (scalar + vector addressing) where vector index register is the same as the destination register and element size = 32. Addressing mode is 32b scaled or unscaled offset.

LDFF1W gather (scalar + vector addressing) where vector index register is the same as the destination register and element size = 32. Addressing mode is 32b scaled or unscaled offset.

ST3[B/H/W/D] contiguous (scalar + scalar addressing).

ST4[B/H/D/W] contiguous (scalar + immediate addressing).

ST4[B/H/D/W] contiguous (scalar + scalar addressing).